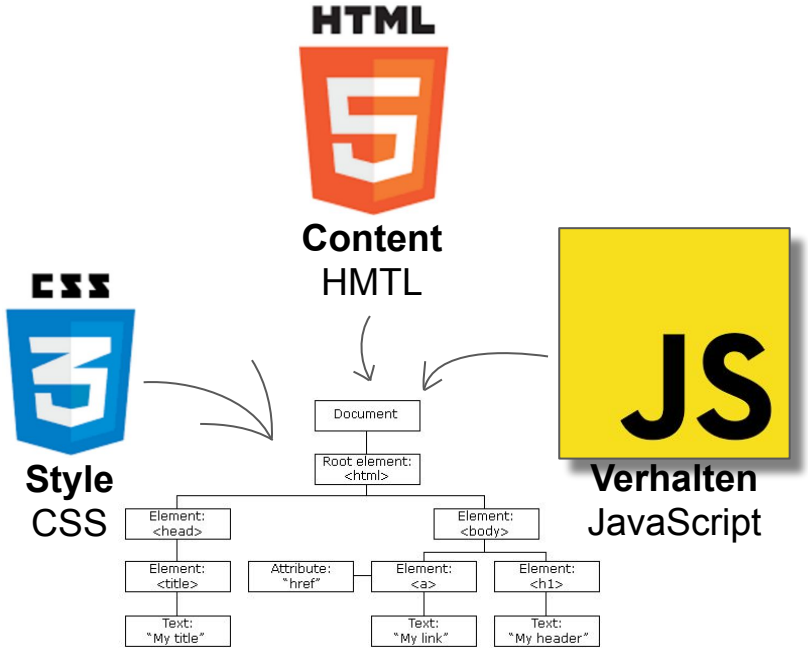
DOM

Document Object Model



Das Zusammenspiel aller drei Technologien

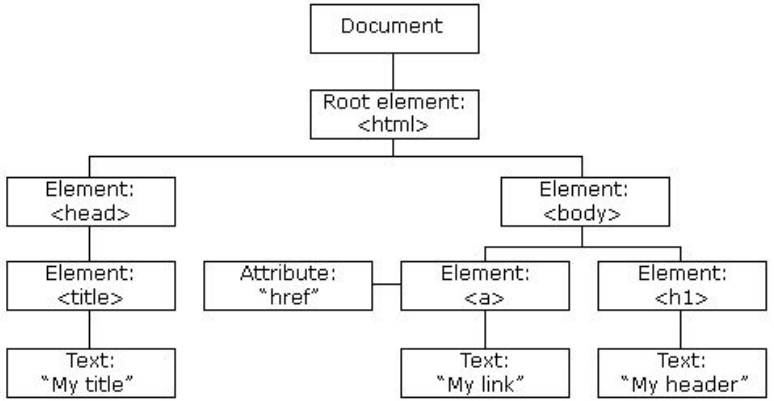
Compilerbau: Parser ⇒ AST

kontextfreie Sprache
(Chomsky-2)

```
<!DOCTYPE html>
<html lang="de">
<head>
    <title>My title</title>
</head>
<body>
   <h1>My header</h1>
   <a href="link.html">My link</a>
</body>
</html>
```

Parser

Abstract Syntax Tree (AST)



Besonderheit: Der AST ist **standardisiert** für HTML und XML und heißt Document Object Model (DOM).

Webentwickler-Humor auf Stackoverflow

36 Answers

Active | Oldest | Votes

1 2 Next

4413

Locked. There are disputes about this answer's content being resolved at this time. It is not currently accepting new interactions.

You can't parse [X]HTML with regex. Because HTML can't be parsed by regex. Regex is not a tool that can be used to correctly parse HTML. As I have answered in HTML-and-regex questions here so many times before, the use of regex will not allow you to consume HTML. Regular expressions are a tool that is insufficiently sophisticated to understand the constructs employed by HTML. HTML is not a regular language and hence cannot be parsed by regular expressions. Regex queries are not equipped to break down HTML

Have you tried using an XML parser instead?

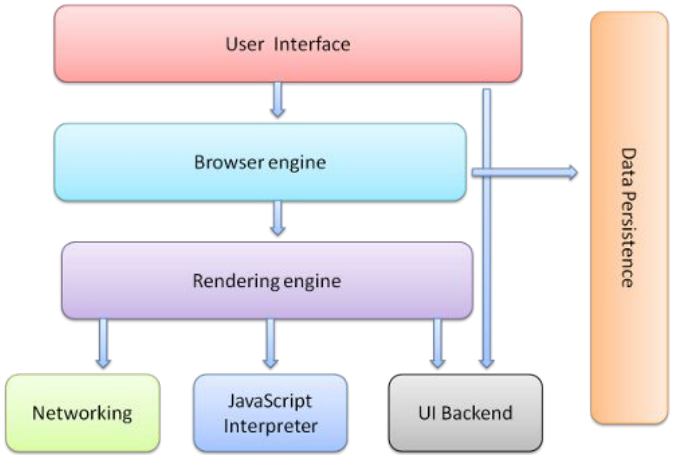
Moderator's Note

This post is locked to prevent inappropriate edits to its content. The post looks exactly as it is supposed to look - there are no problems with its content. Please do not flag it for our attention.

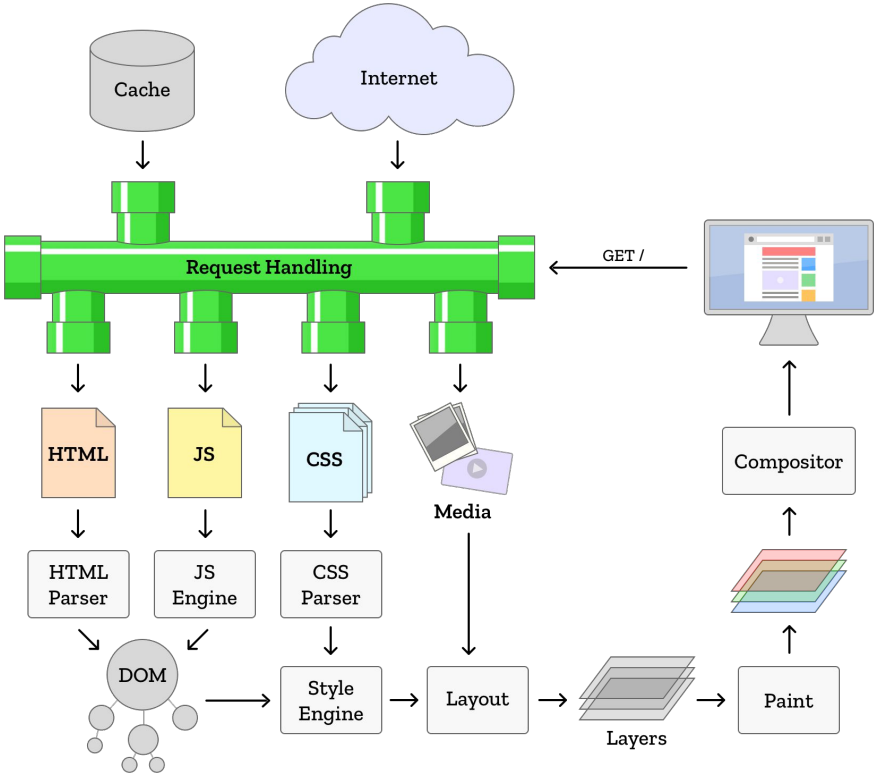
as a tool to process HTML
and the dread realm of co
corrupt) a mere glimpse o
instantly transport a *progr*
ceaseless screaming, he comes, ~~the pestilent slithy~~ regex-infection wil
l devour your HTML parser, application and existence for all time like
Visual Basic only worse *he comes he comes do not fight* h**e comes,**
his *unholy radiancé destroying all enlightenment, HTML tags* ***leaking from your eyes like liquid*** *pain, the song of regular expression* ~~parsing~~ will *extinguish the voices of mortal* ***man from the sphere I*** *can see it can you see it it is beautiful the* `final snuf` *fing of* ***the lies of Man ALL IS LOST ALL IS*** LOST *the pony he comes* he ~~comes he comes~~ ***the ichor*** permeates *all* MY FACE *MY FACE oh god no* ***NO NOOOO*** ~~NO~~ *stop the angles* `are` ***not real ZALGO IS*** TONY THE ~~PONY HE COMES~~

https://stackoverflow.com/questions/1732348/regex-match-open-tags-except-xhtml-self-contained-tags

Gemeinsame Software-Architektur aller Browser



https://www.html5rocks.com/de/tutorials/internals/howbrowserswork/

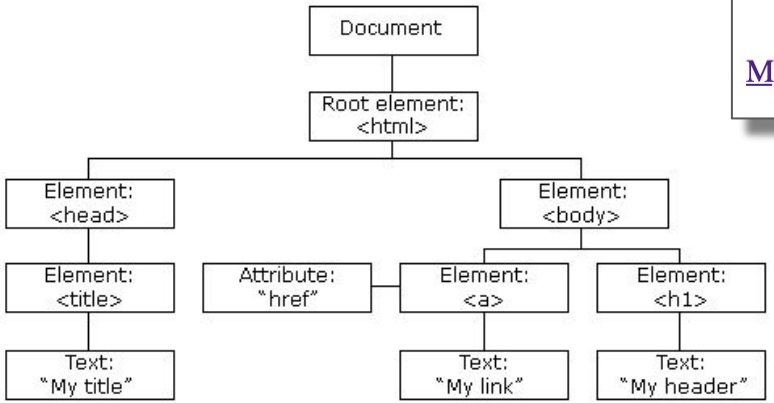


https://hacks.mozilla.org/2017/05/quantum-up-close-what-is-a-browser-engine/

DOM-API

My header

My link



Another header

My link

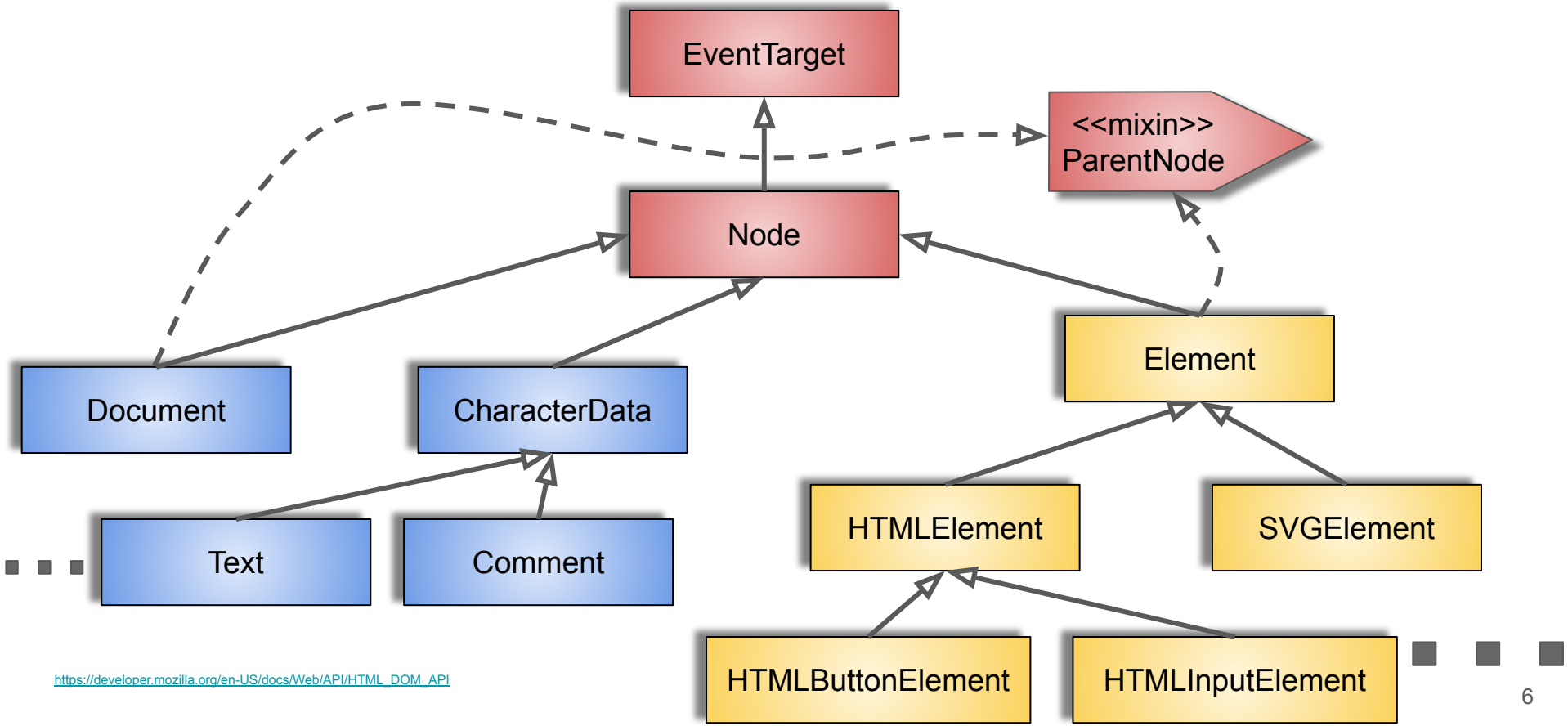
HTML

```
<!DOCTYPE html>
<html lang="de">
<head>
   <title>My title</title>
</head>
<body>
<h1 id="top">My header</h1>
<a id="ref" href="link.html">My link</a>
</body>
</html>
```

```
<script>
   const h1 = document.getElementById('top');
   h1.textContent = "Another header";
   const ref = document.getElementById('ref');
   ref.href = "http://kaul.inf.h-brs.de";
</script>
```

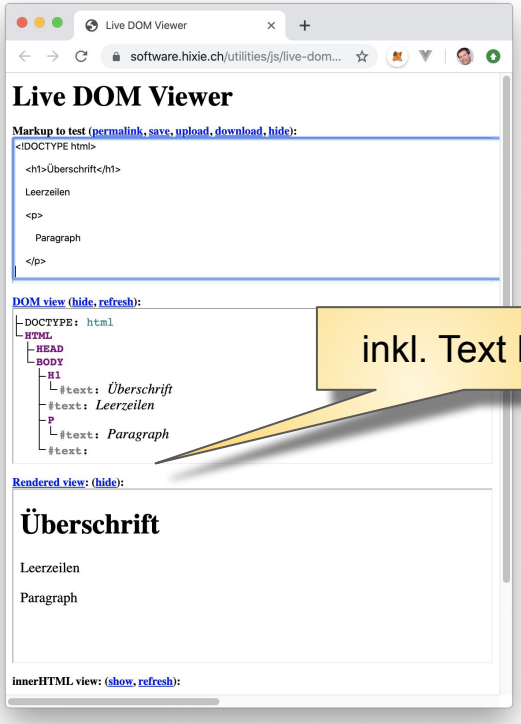
JS

DOM-Knoten-Interface-Vererbungshierarchie

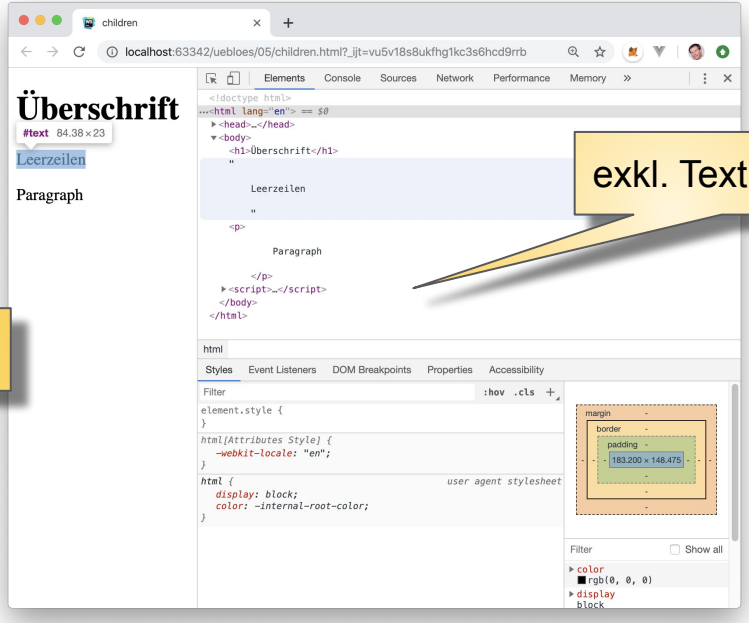


https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API

DOM-Inspektor versus Element-Inspektor



inkl. Text Nodes



exkl. Text Nodes

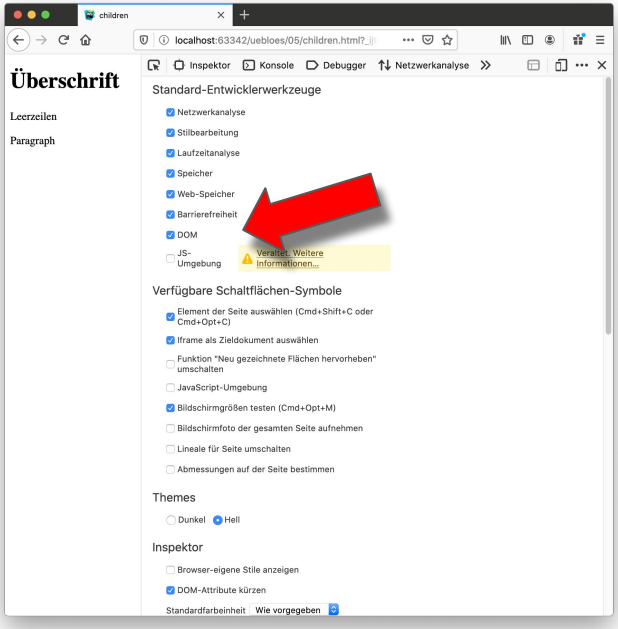
https://software.hixie.ch/utilities/js/live-dom-viewer/

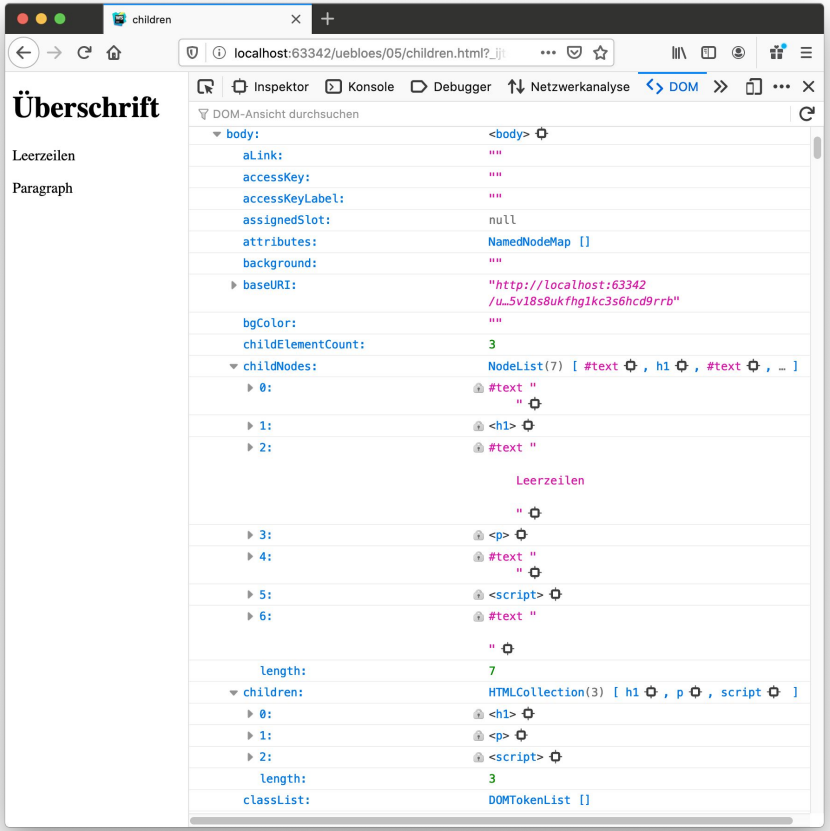
Chrome Elements

- nur Elemente
- keine Text Nodes

Firefox DOM-Inspektor

Einstellungen:





document

Document

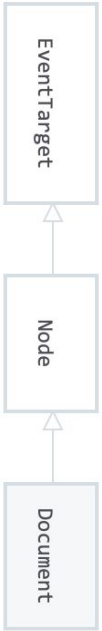
Der Wurzel-Knoten für den Einstieg in den DOM.

Properties

- head
- title
- lastModified
- body
- characterSet
- documentURI
- cookie
- links
- images

Methods

- write()
- getElementById()
- querySelector()
- querySelectorAll()
- getElementsByClassName()
- getElementsByTagName()
- getElementsByName()
- createElement()



https://developer.mozilla.org/en-US/docs/Web/API/document

Erzeugungsmethoden in **document**

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>||Working with elements||</title>
</head>
<body>
  <div id="div1">The text above has been created dynamically.</div>
</body>
</html>
```

JavaScript

```
document.body.onload = addElement;

function addElement () {
  // create a new div element
  // and give it some content
  var newDiv = document.createElement("div");
  var newContent = document.createTextNode("Hi there and greetings!");
  newDiv.appendChild(newContent); //add the text node to the newly created div.

  // add the newly created element and its content into the DOM
  var currentDiv = document.getElementById("div1");
  document.body.insertBefore(newDiv, currentDiv);
}
```

d = **document**

d.createElement	creates the HTML element
d.createTextNode	Creates a new Text node.
var last_p = d.createElement('p'); d.body.appendChild(last_p);	

https://developer.mozilla.org/en-US/docs/Web/API/Document/createElement

DOM - Selektionsmethoden

ed = HTMLElement or Document

document.**getElementById**	Returns a reference to the element by its ID inside document
ed.getElementsByClassName	Returns an array-like object of all child elements which have all of the given class names.
ed.getElementsByTagName	Returns an HTMLCollection of elements with the given tag name.
ed.getElementsByName	Returns a nodelist collection with a given name.
ed.querySelector	Returns the first Element that matches the specified group of CSS selectors
ed.querySelectorAll	Returns a list of the elements that matches the specified group of CSS selectors

Note: The matching is done using **depth-first pre-order** traversal of the document's nodes starting with the first element in the document's markup and iterating through sequential nodes by order of the number of child nodes.

Damit kann man auch **Unterbäume** durchsuchen!

https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector

Problem: HTMLCollection ist kein JS-Array

Lösung ES6-Spread-Operator "…" zur Iteration über HTMLCollection

```
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Spread</title>
7  </head>
8  <body>
9  <h1>Spread Operator</h1>
10 <ul>
11     <li>1</li>
12     <li>2</li>
13 </ul>
14 <script>
15     document.getElementsByTagName('li').forEach( li => li.innerHTML += ". Fall" );
16 </script>
17 <
18 <
```

Spread Operator

- 1. Fall
- 2. Fall

```
HTMLCollection(2)
▶ 0: li
▶ 1: li
  length: 2
▶ __proto__: HTMLCollection
```

```
▶ Uncaught TypeError:
document.getElementsByTagName(...).forEach is not a function
    at Spread.html?_ijt=1p9…omsgh56bp18i07f4:15
```

```
<!doctype html>
<h1>Spread Operator</h1>
<ul>
  <li>1</li>
  <li>2</li>
</ul>
<script>
  [...document.getElementsByTagName('li')].forEach(li=>li.innerHTML += ". Fall");
</script>
```

https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Operators/Spread_operator

Alternativen: DOMCollection ⇒ Array

- Spread-Operator: "..."

```
[...document.getElementsByTagName('li')].forEach( callback );
```

- for … of loop
- Array.from

```
Array.from( document.getElementsByTagName('li') )
```

- Array.prototype.forEach

```
Array.prototype.forEach.call( document.getElementsByTagName('li'),
                                                        callback );
```

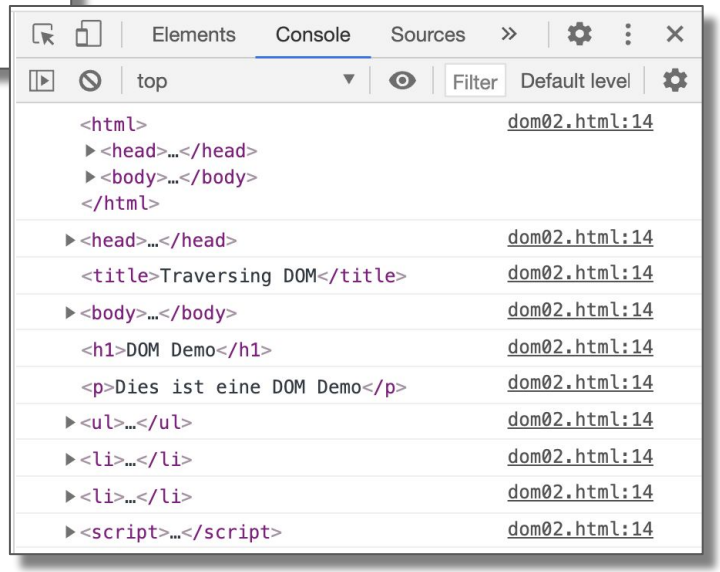
Traversing DOM Elements only

```
<!DOCTYPE html>
<head>
   <title>Traversing DOM</title>
</head>
<body>
   <h1>DOM Demo</h1>
   <p>Dies ist eine DOM Demo</p>
   <ul>
       <li>erste Zeile</li>
       <li>zweite Zeile</li>
   </ul>
   <script>
       for ( const elem of document.querySelectorAll('*') ){
         console.log( elem );
       }
   </script>
</body>
```

DOM Demo

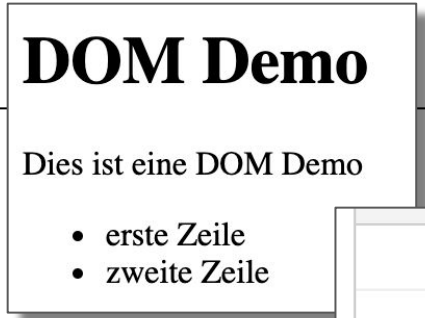
Dies ist eine DOM Demo

- erste Zeile
- zweite Zeile



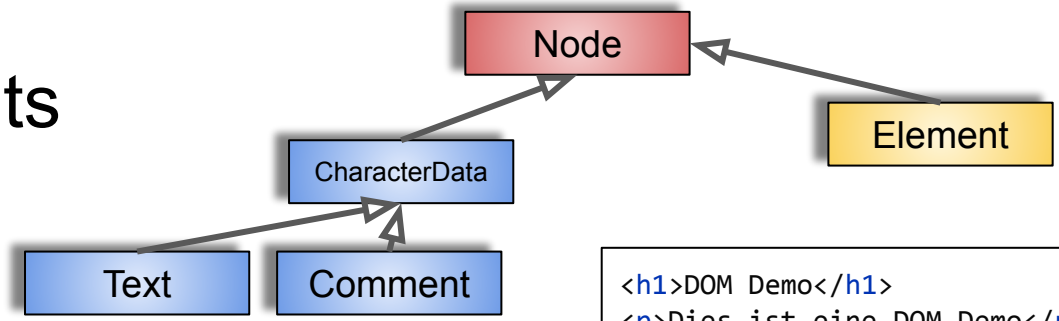
Traversing all child nodes

```
<!DOCTYPE html>
<head>
   <title>Traversing DOM</title>
</head>
<body>
   <h1>DOM Demo</h1>
   <p>Dies ist eine DOM Demo</p>
   <ul>
       <li>erste Zeile</li>
       <li>zweite Zeile</li>
   </ul>
   <script>
       for ( const nodes of document.querySelector('ul').childNodes ){
         console.log( nodes );
       }
   </script>
</body>
```





Nodes vs. Elements




```
<script>
 const traversed = new Set();
 for ( const elem of document.querySelectorAll('*') ){
   traversed.add( elem );
   console.log( elem );
 }
 console.log( "The document has " + traversed.size + " elements." );
 console.log( "=== Text Nodes ===" );
 const textNodes = new Set();
 for ( const node of traversed ){
   for ( const child of node.childNodes ){
     if ( ! traversed.has( child ) ){
       textNodes.add( child )
       console.log( child );
     }
   }
 }
 console.log( "The document has " + textNodes.size + " textNodes." );
</script>
```

```
<h1>DOM Demo</h1>
<p>Dies ist eine DOM Demo</p>
<ul>
   <li>erste Zeile</li>
   <li>zweite Zeile</li>
</ul>
```

The document has 10 elements.
The document has 16 textNodes.
⇒ Sum 26 Nodes

interface Node

Node

<<mixin>>
ParentNode

Node properties:

- childNodes
- firstChild
- lastChild
- nextSibling
- previousSibling
- parentNode
- parentElement
- nodeName
- nodeType
- textContent

Node methods:

- appendChild()
- removeChild()
- replaceChild()
- contains()
- hasChildNodes()
- insertBefore()

ParentNode properties:

- children
- childElementCount
- firstElementChild
- lastElementChild

ParentNode methods:

- append()
- prepend()
- querySelector()
- querySelectorAll()
- replaceChildren()

- nextElementSibling
- previousElementSibling

https://developer.mozilla.org/en-US/docs/Web/API/Node

interface HTMLElement

HTMLElement properties:

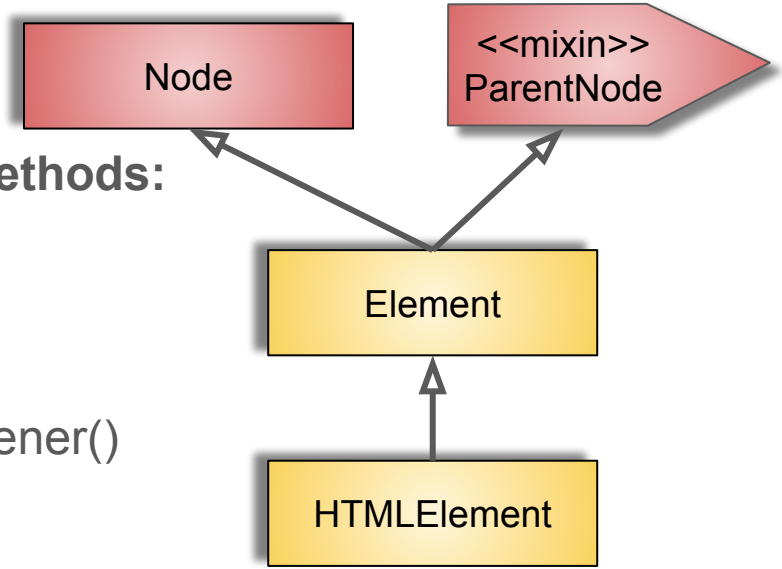
- id
- innerHTML
- outerHTML
- textContent
- innerText
- outerText
- firstElementChild
- lastElementChild
- nextElementChild
- previousElementChild
- children
- contentEditable

■ ■ ■

HTMLElement methods:

- getAttribute()
- setAttribute()
- hasAttribute()
- addEventListener()
- click()
- closest()

■ ■ ■



https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement

Standard

HTMLElement

Web technology for developers › Web APIs › HTMLElement

Jump to: Properties Methods Events Specifications Browser compatibility See also

The `HTMLElement` interface represents any HTML element. Some elements directly implement this interface, while others implement it via an interface that inherits it.



Properties

Inherits properties from its parent, `Element`, and implements those from `GlobalEventHandlers` and `TouchEventHandlers`.

`HTMLElement.accessKey`
Is a `DOMString` representing the access key assigned to the element.

`HTMLElement.accessKeyLabel` Read only
Returns a `DOMString` containing the element's assigned access key.

`HTMLElement.contentEditable`
Is a `DOMString`, where a value of `"true"` means the element is editable and a value of `"false"` means it isn't.

`HTMLElement.isContentEditable` Read only
Returns a `Boolean` that indicates whether or not the content of the element can be edited.

Methods

Inherits methods from its parent, `Element`.

`HTMLElement.attachInternals()`
Attaches an `ElementInternals` instance to the custom element.

`HTMLElement.blur()`
Removes keyboard focus from the currently focused element.

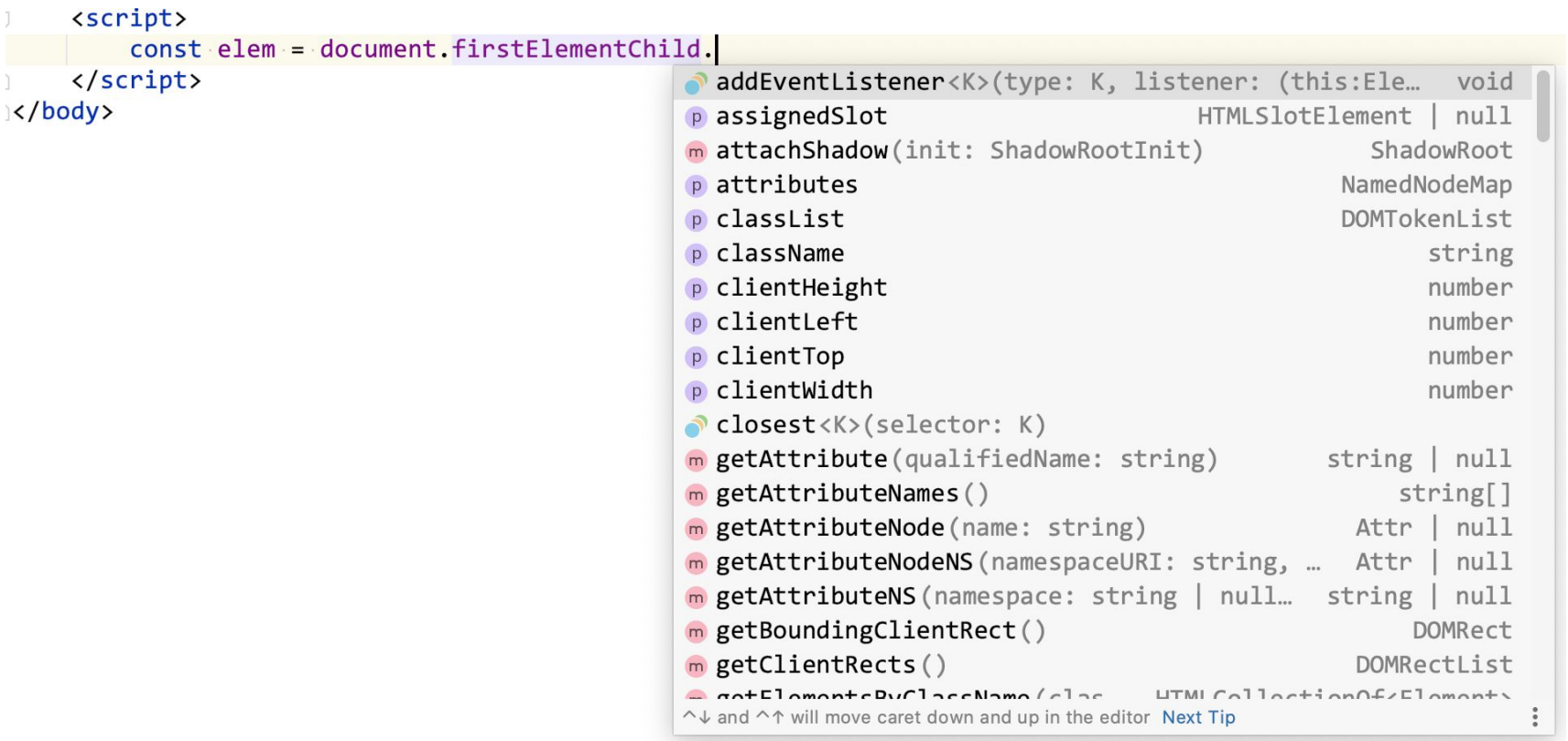
`HTMLElement.click()`
Sends a mouse click event to the element.

`HTMLElement.focus()`
Makes the element the current keyboard focus.

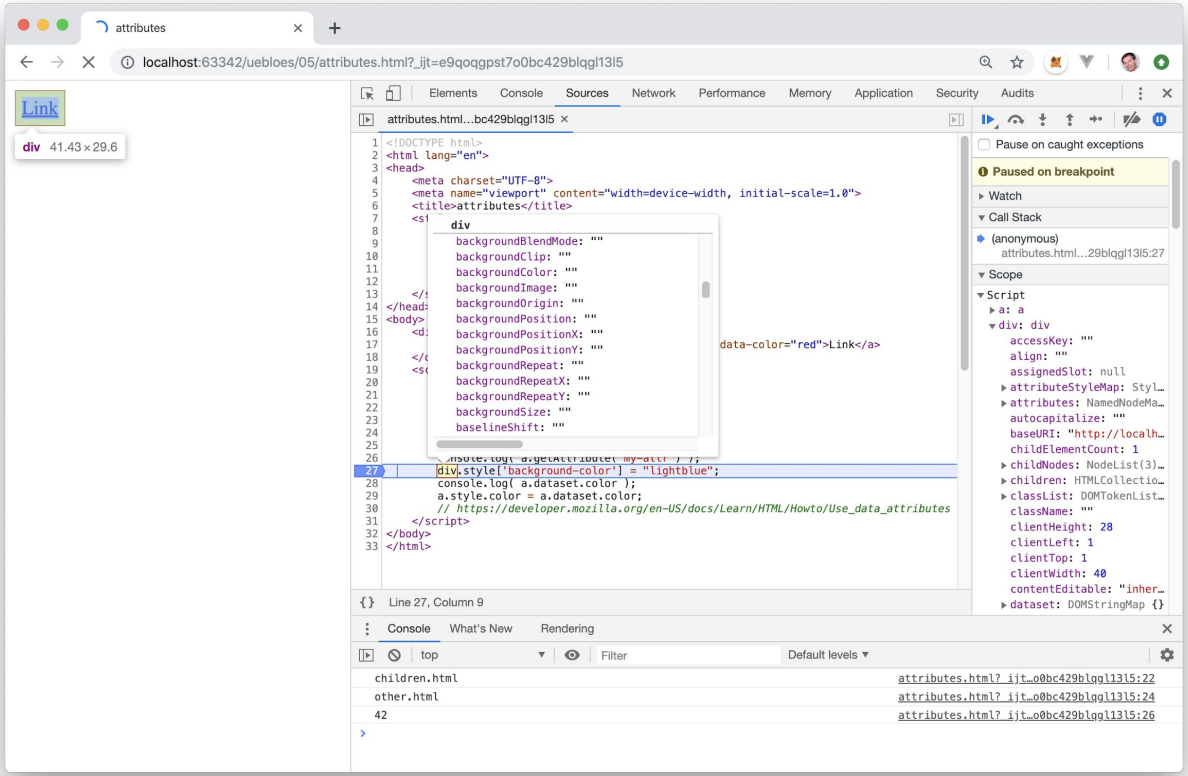
`HTMLElement.forceSpellCheck()`
Runs the spell checker on the element's contents.

https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement

IDE Support Auto-Complete



Demo:
Exploring the Element Interface in DevTools



https://developers.google.com/web/tools/chrome-devtools

Attribute vs. Properties:
2 Welten: HTML versus JavaScript-Objekte

HTML Attributes

Object Properties

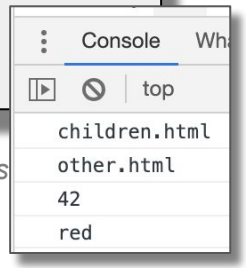
getAttribute('name')

```
<div>
  <a href="children.html" data-color="red">Link</a>
</div>
```

```
<script>
  const div = document.querySelector('div');
  const a = document.querySelector('a');
  console.log( a.getAttribute('href') );
  a.setAttribute( 'href', 'other.html' );
  console.log( a.getAttribute('href') );
  a.setAttribute( 'my-attr', 42 );
  console.log( a.getAttribute('my-attr') );
  div.style['background-color'] = "lightblue";
  console.log( a.dataset.color );
  a.style.color = a.dataset.color;
</script>
```

setAttribute('name')

a.my-attr *// Syntaxfehler, da Minus arithmetische Operation*



https://developer.mozilla.org/en-US/docs/Learn/HTML/Howto/Use_data_attributes
https://developer.mozilla.org/de/docs/Web/JavaScript/Guide/Working_with_objects
https://developer.mozilla.org/en-US/docs/Web/API/Element/getAttribute

Erweiterbarkeit der HTML-Attribute mit data-*
Die **dataset** - Schnittstelle

```
<div>
  <a href="children.html"
    data-color="red"
    data-number="3"
    data-info="any info">Link</a>
</div>
<script>
  const a = document.querySelector('a');

  console.log( a.dataset.color ); // red
  a.style.color = a.dataset.color;
  console.log( a.dataset.number );  // 3
  console.log( a.dataset.info ); // any info

</script>
```

user-defined attributes

In JavaScript Zugriff via **dataset**-Schnittstelle

https://developer.mozilla.org/en-US/docs/Web/API/HTMLOrForeignElement/dataset

https://developer.mozilla.org/en-US/docs/Learn/HTML/Howto/Use_data_attributes

Wie ruft man in JavaScript den HTML-Parser auf?

```
<!DOCTYPE html>
<title>HTML-Parser</title>
<div id="root"></div>
<script>
   const root = document.getElementById('root');
   const count = 5;
   root.innerHTML = `
       <h1>DOM Demo</h1>
       <p>Dies ist eine DOM Demo</p>
       <ul>
          <li>erste Zeile</li>
          ${addItems(count)}
       </ul>
   `;
   function addItems( count ){
     let result = '';
     for (let i=0; i<count; i++){
       result += `<li>${i+2}. Zeile</li>`;
     }
     return result;
   }
</script>
```

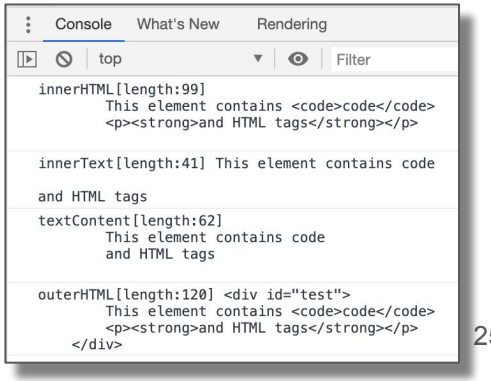


https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML

Lesen mit innerHTML, innerText, textContent und outerHTML

- **innerHTML**
  mit inneren HTML-Tags
- **innerText**
  ohne Tags, style-aware
- **textContent**
  ohne Tags, not aware of style
- **outerHTML**
  mit äußeren Tags

```
  <div id="test">
    This element contains <code>code</code>
    <p><strong>and HTML tags</strong></p>
  </div>
<script>
  const x = document.getElementById('test');
  ['innerHTML', 'innerText', 'textContent', 'outerHTML' ].forEach(
        prop => {
    console.log( `${prop}[length:${x[prop].length}] ${x[prop]}` );
  });
</script>
```



https://stackoverflow.com/questions/19030742/difference-between-innertext-and-innerhtml

Schreiben mit innerHTML, innerText, textContent und outerHTML

- **innerHTML**
  ruft HTML-Parser, Layouter, Painter auf
- **innerText**
  nur Text, **Reflow**, style aware
- **textContent**
  nur Text, not aware of style
- **outerHTML**
  überschreibt auch äußere Tags, ruft HTML-Parser, Layouter, Painter auf

```
<div id="test">
   This element contains <code>code</code>
   <p><strong>and HTML tags</strong></p>
</div>
```

```
<script>
  const testDiv = document.getElementById('test');
  const content = `This element contains <code>code</code>
     <p><strong>and HTML tags</strong></p>`;
  testDiv.innerHTML = content;
  // testDiv.innerText = content;
  // testDiv.textContent = content;
  // testDiv.outerHTML = content;
</script>
```

https://stackoverflow.com/questions/19030742/difference-between-innertext-and-innerhtml

***testDiv*.innerHTML = *content*;**

This element contains code

and HTML tags

```
▼<body>
  ▼<div id="test">
      "This element contains "
      <code>code</code>
    ▼<p>
        <strong>and HTML tags</strong>
      </p>
    </div>
```

***testDiv*.outerHTML = *content*;**




```
▼<body>
    "
        This element contains "
    <code>code</code>
  ▼<p>
      <strong>and HTML tags</strong>
    </p>
```

https://stackoverflow.com/questions/19030742/difference-between-innertext-and-innerhtml

**ohne **

***testDiv*.textContent = *content*;**

This element contains <code>code</code> and HTML tags

```
▼<body>
  ▼<div id="test">
     "This element contains <code>code</code>
             <p><strong>and HTML tags</strong></p>"
   </div>
```

**ohne
**

***testDiv*.innerText = *content*;**

This element contains <code>code</code> and HTML tags

```
▼<body>
  ▼<div id="test">
     "This element contains <code>code</code>"
     <br>
     "          <p><strong>and HTML tags</strong></p>"
   </div>
```

https://stackoverflow.com/questions/19030742/difference-between-innertext-and-innerhtml

Differences textContent versus innerText

Don't get confused by the differences between **`Node.textContent`** and **`HTMLElement.innerText`**. Although the names seem similar, there are important differences:

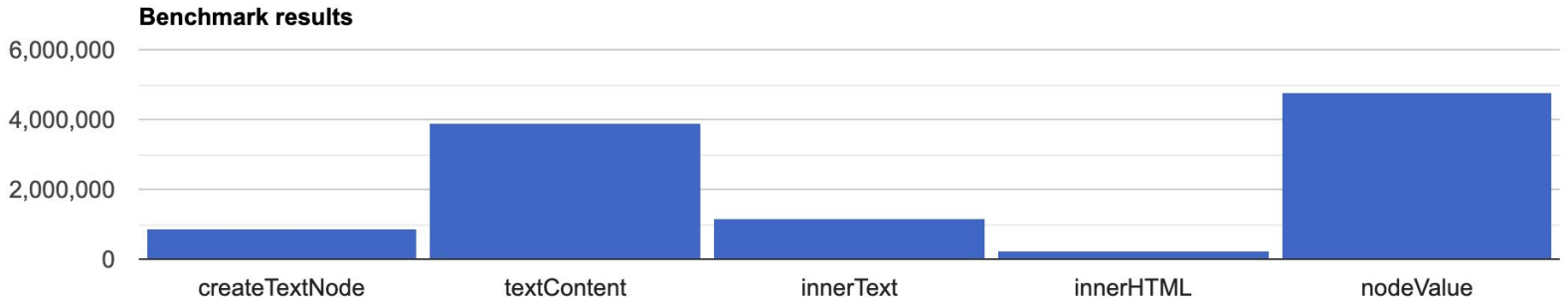
- **`textContent`** gets the content of all elements, **including `<script>` and `<style>`** elements. In contrast, `innerText` only shows “human-readable” elements.
- `textContent` returns every element in the node. In contrast, **`innerText` is aware of styling** and won’t return the text of “hidden” elements. Moreover, since `innerText` takes CSS styles into account, reading the value of **`innerText` triggers a reflow** to ensure up-to-date computed styles. (Reflows can be computationally expensive, and thus should be avoided when possible.)
- Unlike `textContent`, altering `innerText` in **Internet Explorer** (version 11 and below) removes child nodes from the element and *permanently destroys* all descendant text nodes. It is impossible to insert the nodes again into any other element or into the same element anymore.

https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent

Performanz-Vergleich

Test name	Executions per second
createTextNode	896215.1 Ops/sec
textContent	3915397.0 Ops/sec
innerText	1164779.0 Ops/sec
innerHTML	240688.5 Ops/sec
nodeValue	4795645.0 Ops/sec

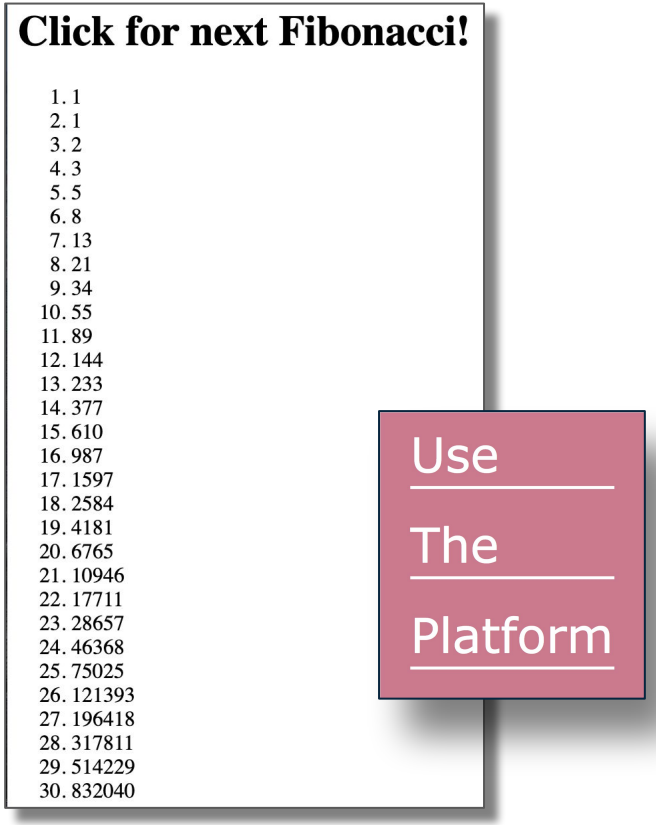
```
const a = document.createElement('a');
a.appendChild(document.createTextNode('text'));
a.textContent = 'text';
a.innerText = 'text';
a.innerHTML = 'text';
a.nodeValue = 'text';
```



https://www.measurethat.net/Benchmarks/Show/7023/0/createtextnode-textcontent-innertext-innerhtml-nodevalu#latest_results_block

appendChild()

```
<!DOCTYPE html>
<h1>Click for next Fibonacci!</h1>
<ol id="list"><li>1</li></ol>
<script>
   const list = document.getElementById('list');
   let fibo = { 0:0n, 1:1n };
   let count = 1;
   document.addEventListener('click', event => {
     count += 1;
     fibo[ count ] = fibo[ count - 1 ] + fibo[ count - 2 ];
     const newItem = document.createElement('li');
     newItem.textContent = fibo[ count ];
     list.appendChild( newItem );
   } );
</script>
```



https://developer.mozilla.org/de/docs/Web/API/Node/appendChild

Übung Web-Entwicklung



Use

The

Platform

3 Methoden, um Event Listeners zu definieren

1. via HTML-Attribut: *inline event handler*

```
<div onclick="changeText(this)">JavaScript wurde noch nicht ausgeführt!</div>
<div id="div2">div2</div>
<div id="div3">div3</div>
<script>
 function changeText( element ){
   element.textContent = "JavaScript wurde jetzt ausgeführt!";
 }

 const div2 = document.getElementById('div2');
 div2.onclick = function( event ){ changeText(this); };

 const div3 = document.getElementById('div3');
 div3.addEventListener('click', function( event ){ changeText(this); } );

 div3.addEventListener( 'click', function( event ){ this.style['background-color'] = '#ffaaaa'; } );

</script>
```

2. via Object Property

3. via Method addEventListener

https://stackoverflow.com/questions/6348494/addeventlistener-vs-onclick

Unobstrusive JavaScript

```
<!DOCTYPE html>
<h1 onclick="this.textContent += 'Ooops!';">Click on this text!</h1>
```

```
<!DOCTYPE html>
<h1 onclick="changeText(this)">Click on this text!</h1>

<script>
 function changeText( elem ) {
   elem.textContent += "Ooops!";
 }
</script>
```

Unobstrusive JavaScript



```
<!DOCTYPE html>
<h1>Click on this text!</h1>

<script>

 const h1 = document.querySelector("h1");
 h1.addEventListener("click", function( event ){
   this.textContent += "Ooops!"
 });

</script>
```

Saubere Trennung von HTML und JavaScript

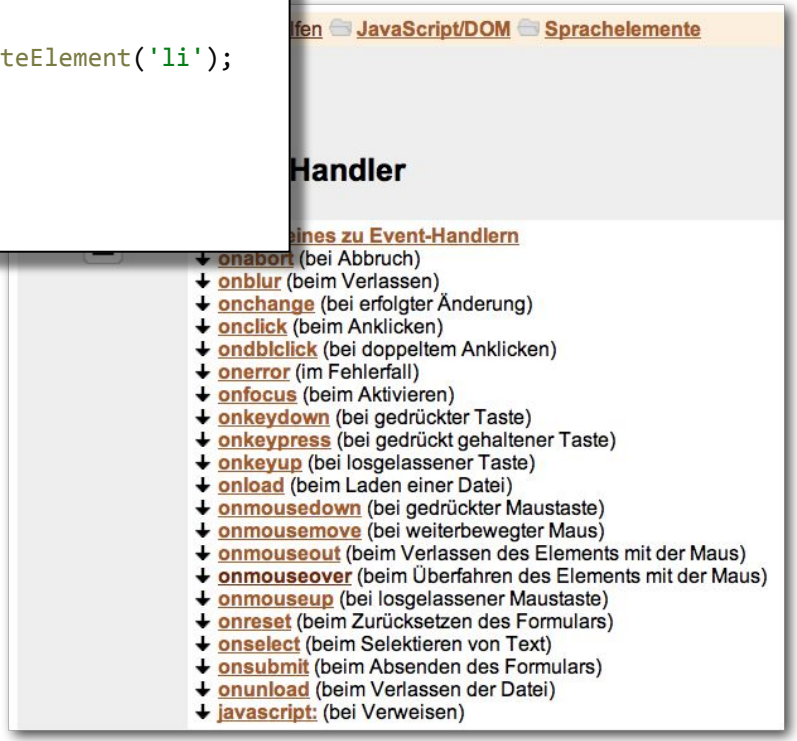
Events ...

Events of DIV

1. onabort
2. onblur
3. oncancel
4. oncanplay
5. oncanplaythrough
6. onchange
7. onclick
8. onclose
9. oncontextmenu
10. oncuechange
11. ondblclick
12. ondrag
13. ondragend
14. ondragenter
15. ondragleave
16. ondragover
17. ondragstart
18. ondrop
19. ondurationchange

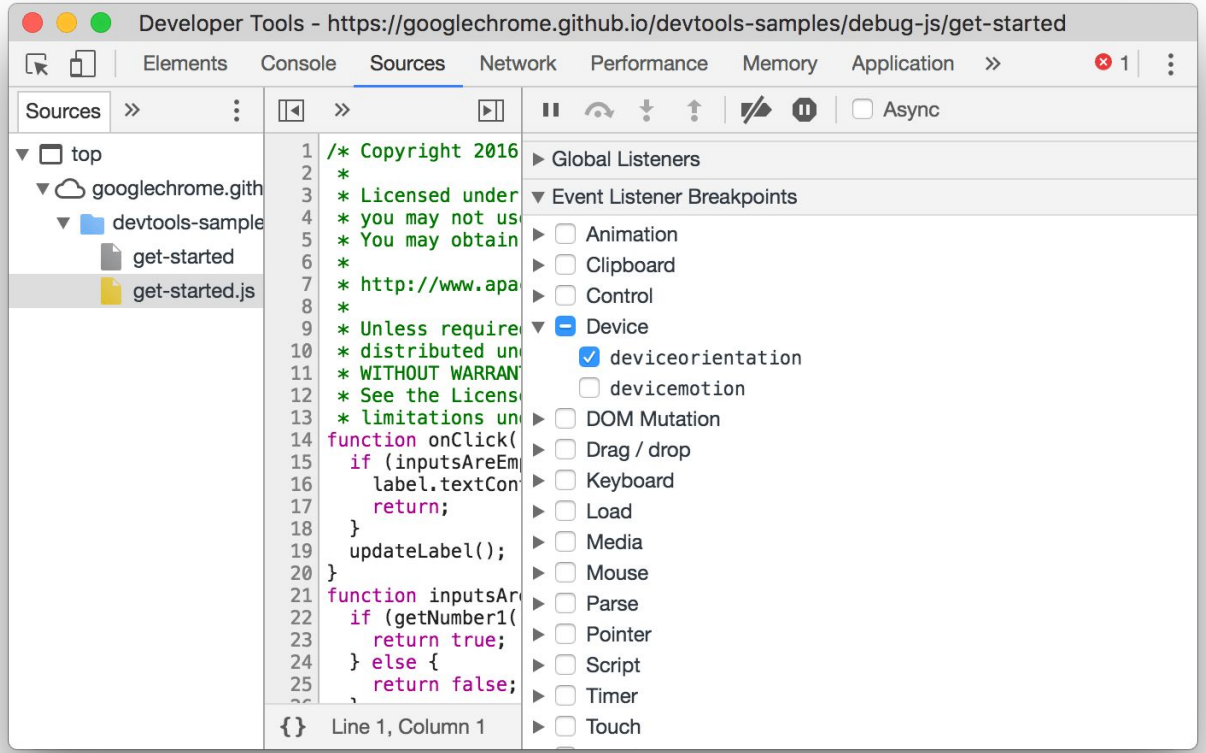
```
<!DOCTYPE html>
<h1>Events of DIV</h1>
<ol id="list"></ol>
<script>
   const list = document.getElementById('list');
   const div = document.createElement( 'div' );
   for ( const prop in div ) {
     if ( prop.startsWith('on') ) {
       const newItem = document.createElement('li');
       newItem.textContent = prop;
       list.appendChild( newItem );
     }
   }
</script>
```

82. oncopy
83. oncut
84. onpaste
85. onoverscroll
86. onscrollend
87. onpointerrawupdate
88. onbeforecopy
89. onbeforecut
90. onbeforepaste
91. onsearch
92. onfullscreenchange
93. onfullscreenerror
94. onwebkitfullscreenchange
95. onwebkitfullscreenerror
96. onbeforexrselect
97. onbeforematch



de.selfhtml.org/javascript/sprache/eventhandler.htm

DevTools Event Breakpoints

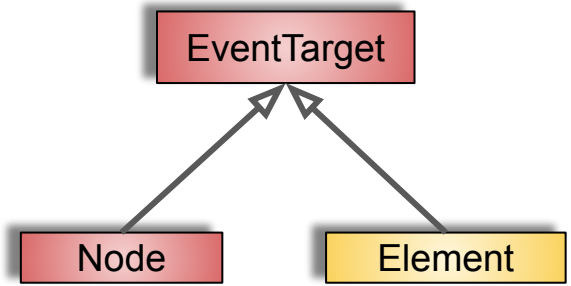


https://developers.google.com/web/tools/chrome-devtools/javascript/breakpoints

EventTarget.addEventListener()

The `EventTarget` method **`addEventListener()`** sets up a function that will be called whenever the specified event is delivered to the target. Common targets are `Element`, `Document`, and `Window`, but the target may be any object that supports events (such as `XMLHttpRequest`).

`addEventListener()` works by adding a function or an object that implements `EventListener` to the list of event listeners for the specified event type on the `EventTarget` on which it's called.



Syntax

```
target.addEventListener(type, listener[, options]);

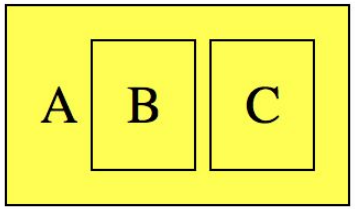
target.addEventListener(type, listener[, useCapture]);
```

https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

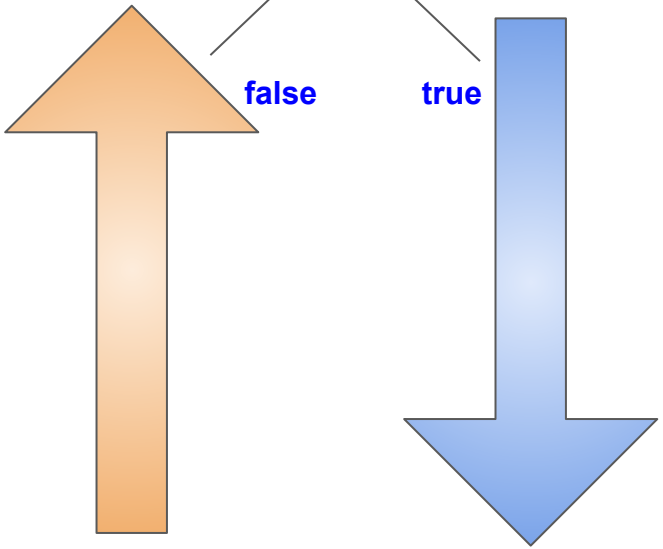
https://wiki.selfhtml.org/wiki/JavaScript/DOM/EventTarget/addEventListener

Bubbling & Capturing

```
addEventListener(event, function, useCapture);
```




```
<div id="a">A
  <div id="b">B</div>
  <div id="c">C</div>
</div>
```



Bubbling

useCapture = false

default

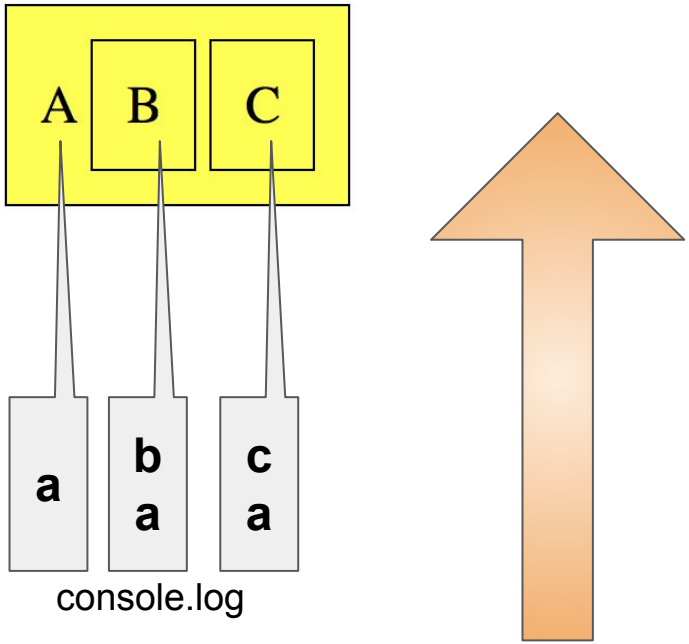
Capturing

useCapture = true

Event Bubbling

```
<!DOCTYPE html>
<style>
   div {
       background-color: yellow;
       border: thin solid black;
       padding: 10px;
       display: inline-block;
   }
</style>
<div id="a">A
   <div id="b">B</div>
   <div id="c">C</div>
</div>
<script>
 for ( const div of document.querySelectorAll('div') ){
   div.addEventListener( 'click', log, false );
 }
 function log(){ console.log( this.id ) }
</script>
```

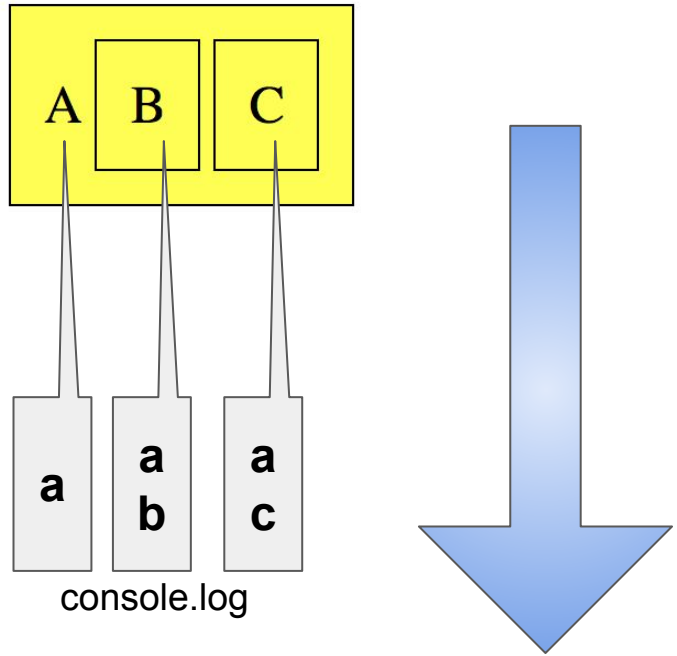
useCapture = false



Event Capturing

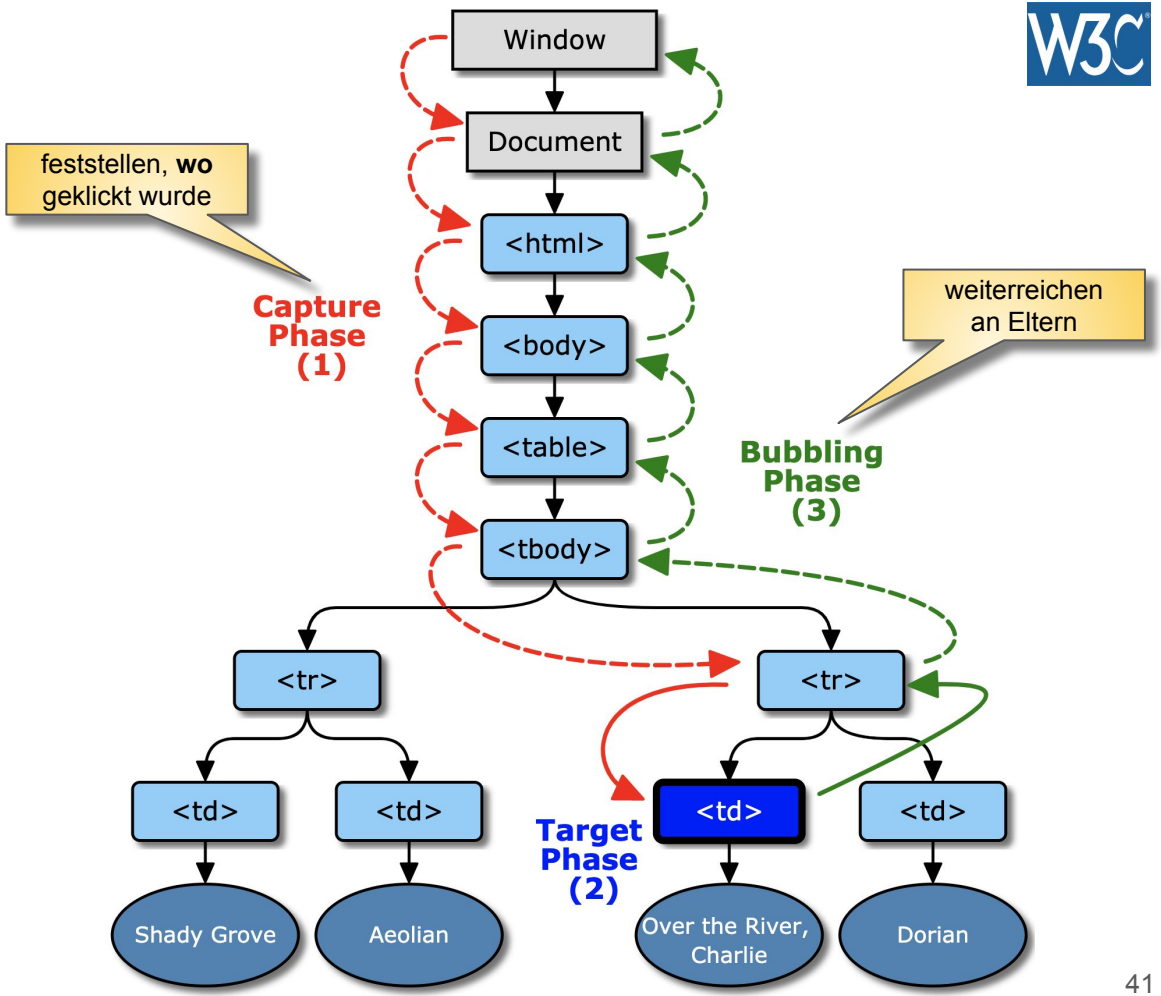
```
<!DOCTYPE html>
<style>
   div {
       background-color: yellow;
       border: thin solid black;
       padding: 10px;
       display: inline-block;
   }
</style>
<div id="a">A
   <div id="b">B</div>
   <div id="c">C</div>
</div>
<script>
 for ( const div of document.querySelectorAll('div') ){
   div.addEventListener( 'click', log, true );
 }
 function log(){ console.log( this.id ) }
</script>
```

useCapture = true



Event Flow

- The **capture phase**: The event object propagates through the target's ancestors from the Window to the target's parent. This phase is also known as the ***capturing phase***.
- The **target phase**: The event object arrives at the event object's event target. This phase is also known as the *at-target phase*. If the event type indicates that the event doesn't bubble, then the event object will halt after completion of this phase.
- The **bubble phase**: The event object propagates through the target's ancestors in reverse order, starting with the target's parent and ending with the Window. This phase is also known as the ***bubbling phase***.



https://www.w3.org/TR/uievents/#event-flow

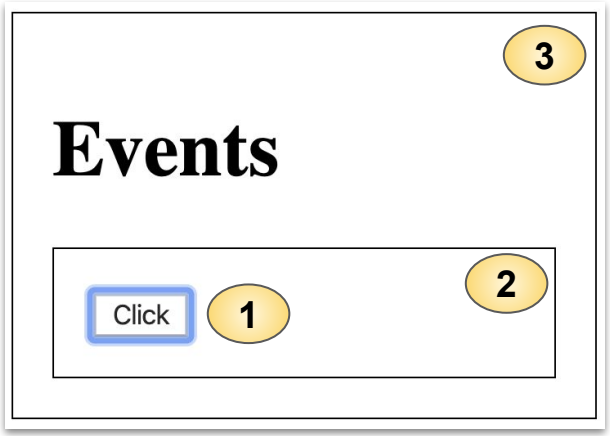
Bubbling Phase

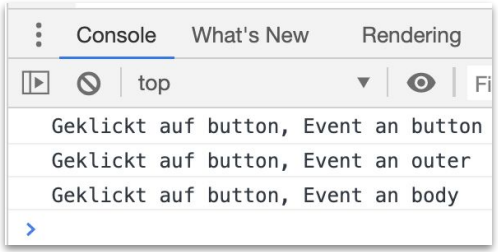
```
<body id="body">
  <h1>Events</h1>
  <div id="outer">
    <button id="button">Click</button>
  </div>

<script>
  function logger( event ){
      console.log( `Geklickt auf ${event.target.id},
                    Event an ${this.id}` ) }

  ['body','outer','button']
    .map( id => document.getElementById( id ) )
    .forEach( elem => elem.addEventListener('click', logger ));

</script>
</body>
```





https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

Capturing Phase

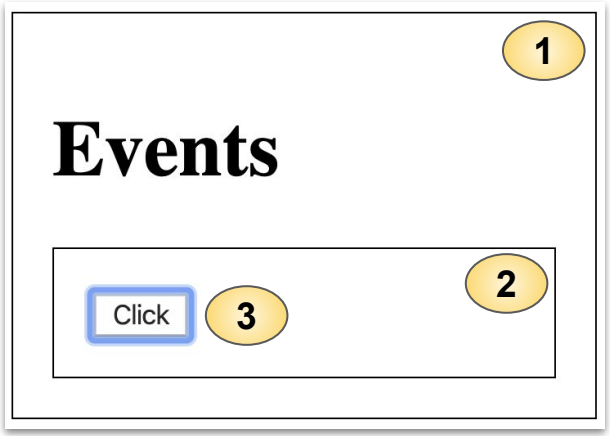
```
<body id="body">
  <h1>Events</h1>
  <div id="outer">
    <button id="button">Click</button>
  </div>

<script>
  function logger( event ){
      console.log( `Geklickt auf ${event.target.id},
                    Event an ${this.id}` ) }

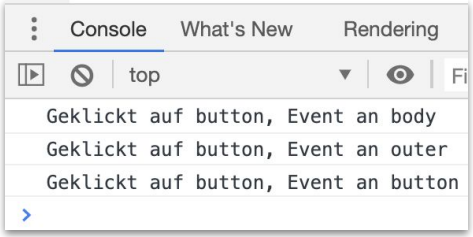
  ['body','outer','button']
    .map( id => document.getElementById( id ) )
    .forEach( elem => elem.addEventListener('click', logger, true ));

</script>
</body>
```

false default







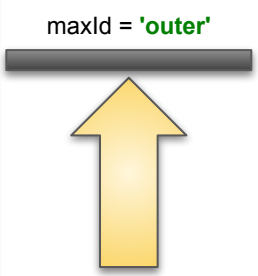
https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener

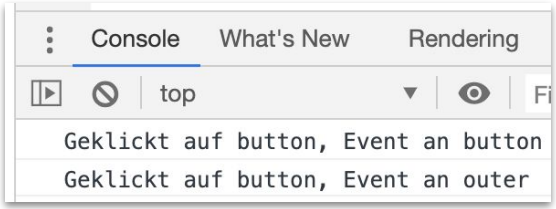
Event.stopPropagation()

```
<body id="body">
  <h1>Events</h1>
  <div id="outer">
    <button id="button">Click</button>
  </div>
<script>
  const stoppingLogger = maxId => {
    return function ( event ) {
      if (this.id === maxId) event.stopPropagation();
      console.log(`Geklickt auf ${event.target.id}, Event an ${this.id}`)
    }
  };

  ['body','outer','button']
    .map( id => document.getElementById( id ) )
    .forEach( elem => elem.addEventListener('click', stoppingLogger( 'outer' ) ));
</script>
</body>
```







https://developer.mozilla.org/en-US/docs/Web/API/Event/stopPropagation

Event.stopImmediatePropagation()

```
<!DOCTYPE html>
<body id="body">
<div id="outer">
   <button id="button">Click</button>
</div>
<script>
 const stoppingLogger = maxId => {
   return function ( event ) {
     if (this.id === maxId) event.stopImmediatePropagation();
     console.log(`Geklickt auf ${event.target.id}, Event an ${this.id},
                                          Phase ${phase(event.eventPhase)}`)
   }
 };

 ['body','outer','button']
 .map( id => document.getElementById( id ) )
 .forEach( elem => elem.addEventListener('click', stoppingLogger( 'outer' ) ));

 for ( const elem of document.querySelectorAll('*') ){
   elem.addEventListener( 'click', log );
 }
 for ( const elem of document.querySelectorAll('*') ){
   elem.addEventListener( 'click', log, true );
 }
 function log( event ){
    console.log( `${this.id} in ${phase(event.eventPhase)}`
 ) }
 function phase( nr ){
   const phases = [ 'NONE', 'CAPTURE', 'TARGET', 'BUBBLE' ];
   return phases[nr];
 }
</script>
</body>
```

event.stopPropagation()

```
 in CAPTURE
body in CAPTURE
outer in CAPTURE
Geklickt auf button, Event an button, Phase TARGET
button in TARGET
button in TARGET
Geklickt auf button, Event an outer, Phase BUBBLE
outer in BUBBLE
```

event.stop**Immediate**Propagation()

```
 in CAPTURE
body in CAPTURE
outer in CAPTURE
Geklickt auf button, Event an button, Phase TARGET
button in TARGET
button in TARGET
Geklickt auf button, Event an outer, Phase BUBBLE
```

https://developer.mozilla.org/en-US/docs/Web/API/Event/stopImmediatePropagation

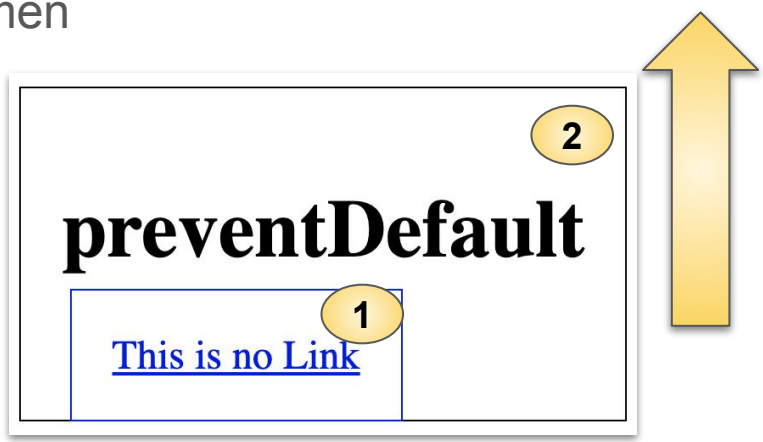
Ein Link soll sich nicht wie ein Link verhalten:

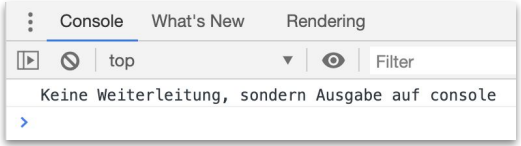
Event.preventDefault()

- das Default-Verhalten des Browsers abklemmen

```
<body>
<h1>preventDefault</h1>
<a href="other.html">This is no Link</a>
<script>
  const a = document.querySelector('a');
  a.addEventListener('click', listener);
  function listener( event ){
    event.preventDefault();
    console.log( 'Keine Weiterleitung, sondern Ausgabe auf console' );
  }
</script>
</body>
```

Event Bubbling findet trotzdem statt.





https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault

return false in EventListener

```
<!DOCTYPE html>
<!-- An inline event handler. -->
<a href="https://google.com" onclick="return false">Google1</a>
<a href="https://google.com" id="ref2">Google2</a>
<script>

 const link = document.getElementById('ref2');

 // JS event handler.
 link.onclick = function( event ) {
   return false;
 };

 // A native event handler.
 link.addEventListener('click', function( event ) {
   return false;
 });

</script>
```

1

2

3

1. Der Link wird nicht aufgerufen, die Google-Seite wird nicht geöffnet, aber das Event breitet sich im Dokument weiter aus.

2. Der Link wird nicht aufgerufen, die Google-Seite wird nicht geöffnet, aber das Event wird gestoppt, also breitet sich im Dokument *nicht* weiter aus.

3. Der Link wird aufgerufen und das Event breitet sich weiter aus. Der EventHandler macht gar nichts. Er kann ersatzlos gestrichen werden.

https://css-tricks.com/dangers-stopping-event-propagation/

Event.data

```
<!doctype html>
<article>
   <h1>Title</h1>
   <textarea></textarea>
</article>

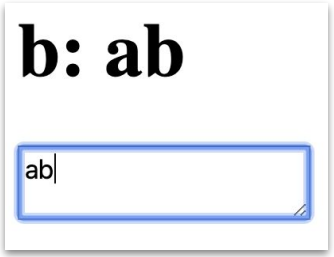
<script>
 const textarea = document.querySelector('textarea');

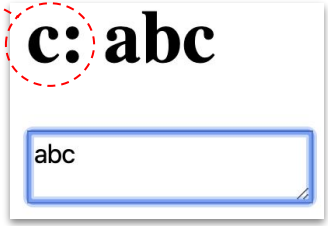
 textarea.addEventListener("input", function( event ){
   this.parentElement.querySelector('h1')
       .textContent = event.data + ": " + this.value;
 });
</script>
```



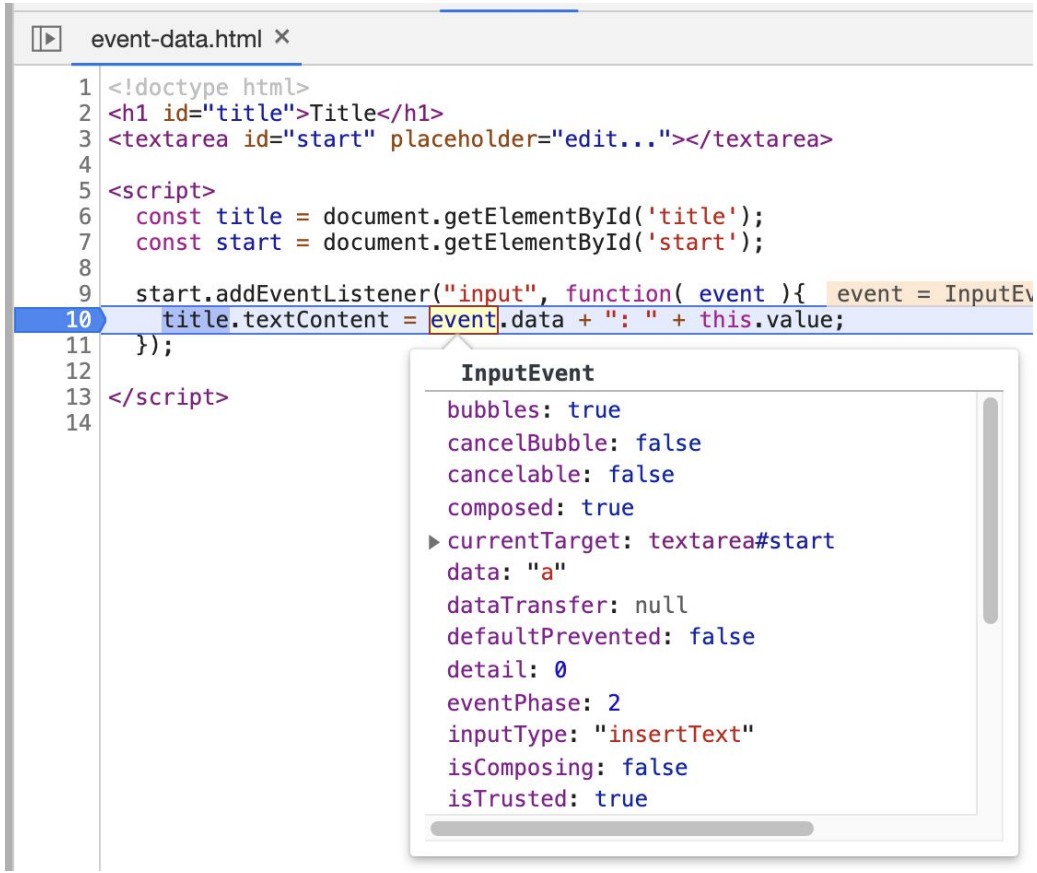
der zuletzt getippte Buchstabe







Inspect Event in DevTools



benutzerdefinierte *CustomEvent*

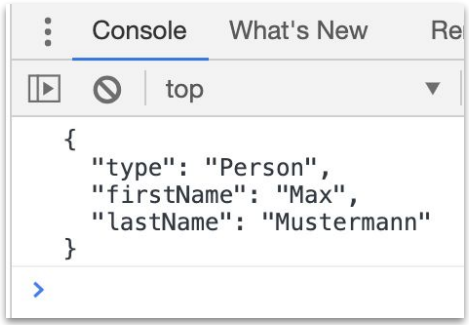
```
<body>
<div id="person"></div>
<script>
  let element = document.getElementById( 'person' );
  let event = new CustomEvent( 'new person', {
    detail: {
      type: 'Person',
      firstName: 'Max',
      lastName: 'Mustermann'
    }
  });
  element.addEventListener( 'new person', logger );
  element.dispatchEvent( event );

  function logger( event ){
    console.log( JSON.stringify( event.detail, null, 2 ) );
  }
</script>
</body>
```

beliebige Datenobjekte im Feld "detail"

benutzerdefinierter Event-Typ "new person"

universelle Ausgabefunktion für beliebig geschachtelte Objekte

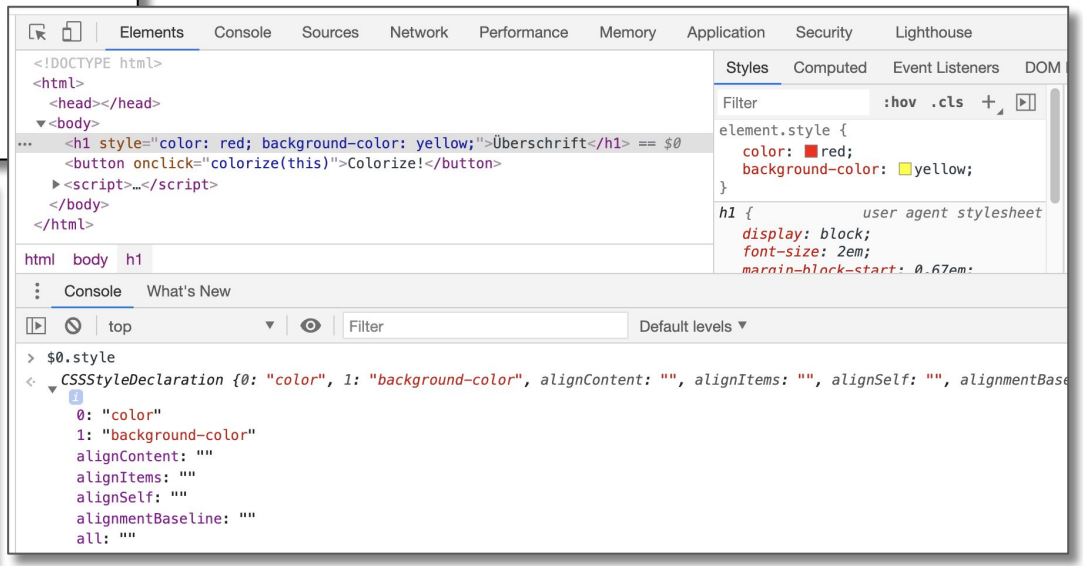


https://developer.mozilla.org/en-US/docs/Web/API/CustomEvent/CustomEvent

Das DOM-StilObjekt: element.style

```
<!DOCTYPE html>
<h1>Überschrift</h1>
<button onclick="colorize(this)">Colorize!</button>
<script>
 function colorize( button ){
   const element = button.previousElementSibling;
   element.style.color = 'red';
   element.style.backgroundColor = "yellow";
 }
</script>
```





zilla.org/en-US/docs/Web/API/CSS_Object_Model/Using_dynamic_styling_information

Stil mit Klasse !

$\Rightarrow$ *Trennung von Layout und JS*

```
<!DOCTYPE html>
<style>
   .colorized {
       color: red;
       background-color: yellow;
   }
</style>
<h1>Überschrift</h1>
<button onclick="colorize(this)">Colorize!</button>
<script>
 function colorize( button ){
   const element = button.previousElementSibling;
   element.classList.toggle("colorized");
 }
</script>
```

https://www.w3schools.com/howto/howto_js_toggle_class.asp

Ein- und Ausblenden von Elementen

display: none

Das HTML-Element wird ganz aus der Seite herausgenommen, verbleibt aber im DOM. Es wird nicht angzeigt und verbraucht auch keinen Platz auf der Seite, aber im Hauptspeicher als Datenstruktur.

visibility: hidden

Das HTML-Element verbleibt auf der Seite und im DOM,
ist allerdings selbst nicht sichtbar.

Es verbraucht Platz und hat Einfluss auf die Platzierung der Nachbar-Elemente.

Mit CSS-Transitionen kann man einen sanften Übergang zwischen sichtbar und unsichtbar erreichen.

https://stackoverflow.com/questions/133051/what-is-the-difference-between-visibilityhidden-and-displaynone

4 Wege, JavaScript in HTML einzubetten

4 Wege der Einbettung:

1. JavaScript direkt hinein schreiben

2. JavaScript-Programm ausführen, um ein anderes JavaScript-Programm aus einer externen Datei zu laden

3. JavaScript aus einer externen Datei laden

4. Inline JavaScript

```
<!doctype html>
<html>
<head>
  <title>My title</title>
  <script> alert('Hallo Welt'); //... JavaScript direkt
    document.write('<script src="http://URL.de"></script>');
  </script>
  <script src="/Pfad/zur/externen_Datei/javascript.js"></script>
</head>
<body>
  <h1>My header</h1>
  <div onclick="alert('Hallo Welt')"> ... </div>
```

JavaScript disabled:

```
<noscript>
  Sorry but you need to have scripting enabled to use this site.
</noscript>
```

<script>-Tag

```
<!doctype html>
<meta charset="utf-8">
<script>
  const myDiv = document.getElementById("myDiv");
  myDiv.textContent = "JavaScript wurde jetzt ausgeführt!"
</script>
<div id="myDiv">JavaScript wurde noch nicht ausgeführt!</div>
```

JavaScript wurde noch nicht ausgeführt!

```
Uncaught TypeError: Cannot set property 'innerHTML' of null
```

Reihenfolge wichtig: DOM-Elemente müssen vorher existieren:

```
<!doctype html>
<meta charset="utf-8">
<div id="myDiv">JavaScript wurde noch nicht ausgeführt!</div>
<script>
  const myDiv = document.getElementById("myDiv");
  myDiv.textContent = "JavaScript wurde jetzt ausgeführt!"
</script>
```

JavaScript wurde jetzt ausgeführt!

Reihenfolge: Ablaufmodell, wie der Browser eine Webseite abarbeitet

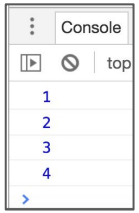
Zeile für Zeile

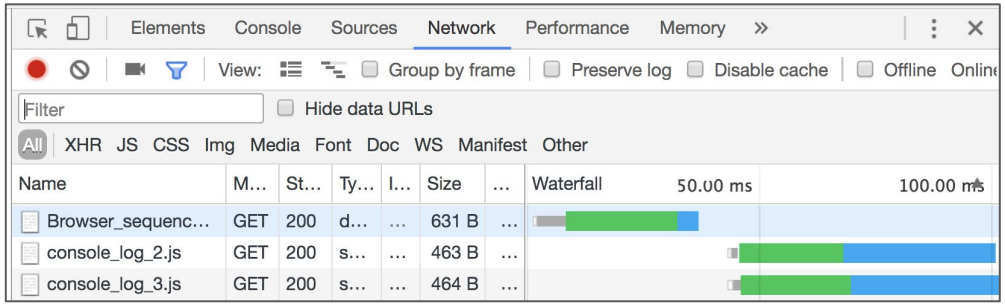
sequentiell

synchron

```
<!doctype html>
<html>
<head>
  <script>console.log( 1 )</script>
  <script src="console_log_2.js"></script>
</head>
<body>
<h1>Browser Ablaufmodell</h1>
<script src="console_log_3.js"></script>
<script>console.log( 4 )</script>
```

Diese Datei wird erst vollständig geladen und ausgeführt, bevor die nächste Zeile angegangen wird.





console_log_2.js

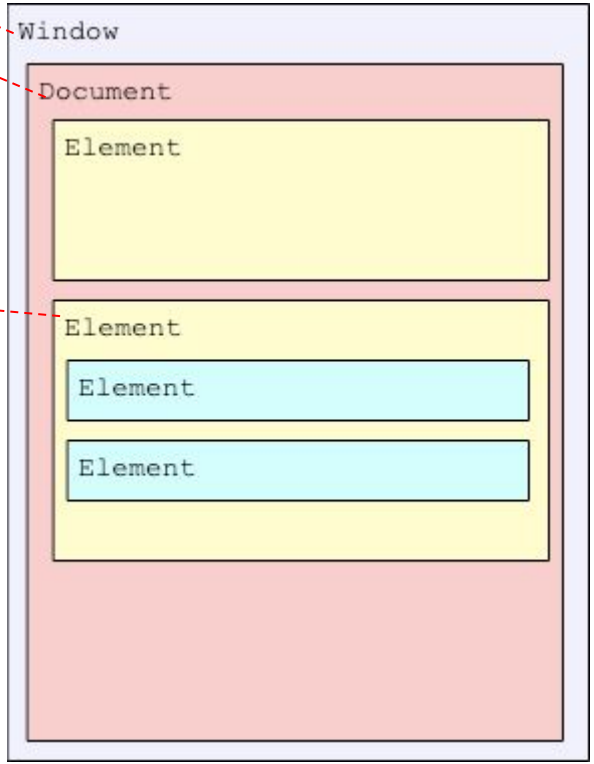
```
console.log(2);
```

`window` ist der globale Namensraum
`document` ist das Wurzelelement des DOM

Namensraum pro Tabulator in Browser

Einstieg in DOM über document

hierarchisch geschachtelte HTMLElemente



https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API

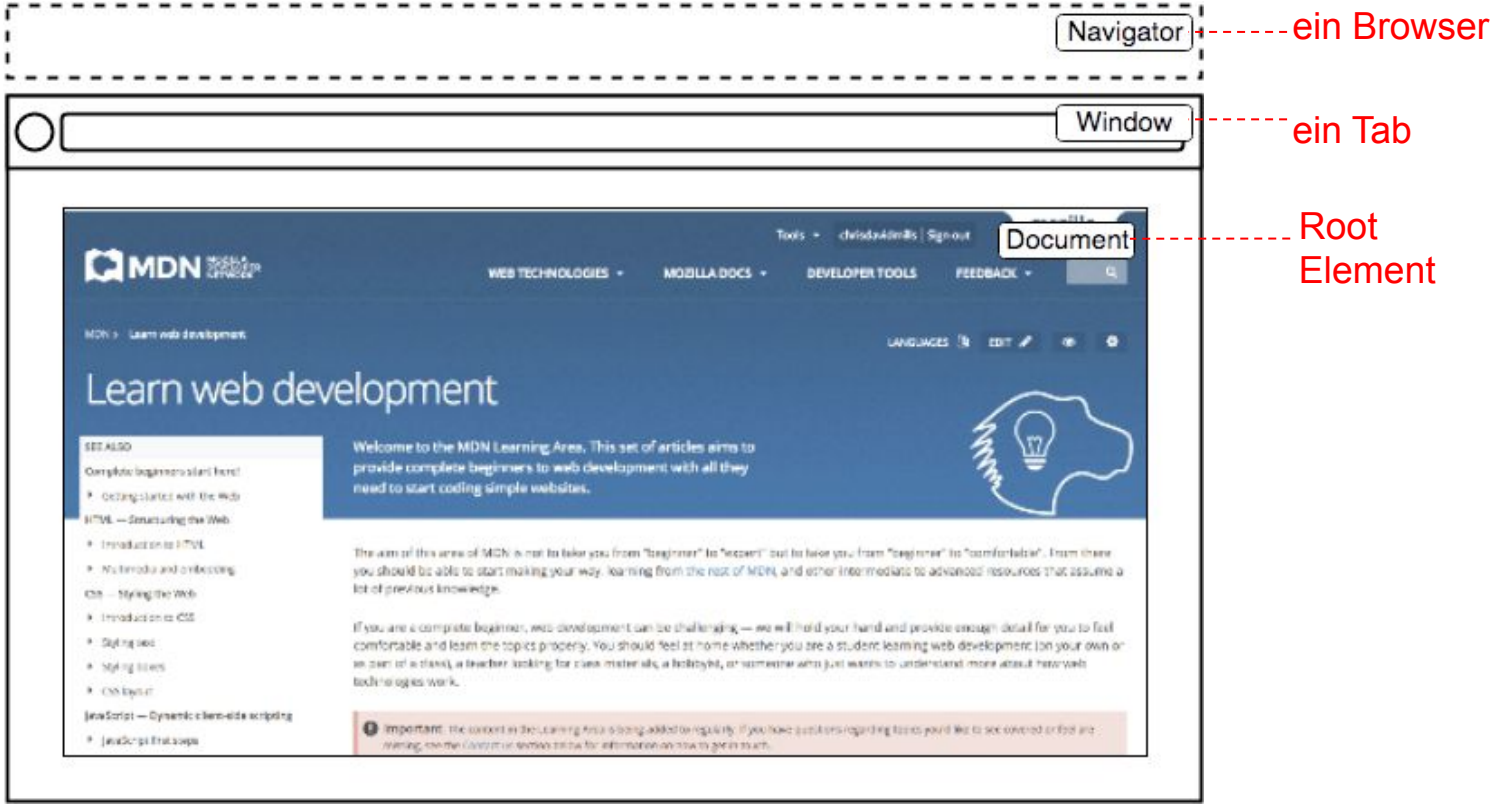
`Object.keys(window)`

```
["InstallTrigger", "alert", "applicationCache", "atob", "blur", "btoa", "caches",
"cancelAnimationFrame", "captureEvents", "clearInterval", "clearTimeout",
"close", "closed", "confirm", "content", "createImageBitmap", "crypto",
"customElements", "devicePixelRatio", "document", "dump", "external",
"fetch", "find", "focus", "frameElement", "frames", "fullScreen",
"getComputedStyle", "getDefaultComputedStyle", "getSelection", "history",
"indexedDB", "innerHeight", "innerWidth", "isSecureContext", "length",
"localStorage", "location", "locationbar", "matchMedia", "menubar", "moveBy",
"moveTo", "mozInnerScreenX", "mozInnerScreenY", "mozPaintCount", "name",
"navigator", "onabort", "onabsolutedeviceorientation", "onafterprint",
"onanimationend", "onanimationiteration", "onanimationstart", "onauxclick",
"onbeforeprint", "onbeforeunload", "onblur", "oncanplay", "oncanplaythrough",
"onchange", "onclick", "onclose", "oncontextmenu", "ondblclick", "ondevicelight",
"ondevicemotion", "ondeviceorientation", "ondeviceproximity", "ondrag",
"ondragend", "ondragenter", "ondragexit", "ondragleave", "ondragover",
"ondragstart", "ondrop", "ondurationchange", "onemptied", "onended", "onerror",
"onfocus", "onhashchange", "oninput", "oninvalid", "onkeydown", "onkeypress",
"onkeyup", "onlanguagechange", "onload", "onloadeddata", "onloadedmetadata",
"onloadend", "onloadstart", "onmessage", "onmousedown", "onmouseenter",
"onmouseleave", "onmousemove", "onmouseout", "onmouseover", "onmouseup",
"onmozfullscreenchange", "onmozfullscreenerror", "onoffline", "ononline",
"onpagehide", "onpageshow", "onpause", "onplay", "onplaying", "onpopstate",
"onprogress", "onratechange", "onreset", "onresize", "onscroll", "onseeked",
"onseeking", "onselect", "onselectstart", "onshow", "onstalled", "onstorage",
"onsubmit", "onsuspend", "ontimeupdate", "ontoggle", "ontransitioncancel",
"ontransitionend", "ontransitionrun", "ontransitionstart", "onunload",
"onuserproximity", "onvolumechange", "onwaiting", "onwebkitanimationend",
"onwebkitanimationiteration", "onwebkitanimationstart", "onwebkittransitionend",
"onwheel", "open", "opener", "outerHeight", "outerWidth", "pageXOffset",
"pageYOffset", "parent", "performance", "personalbar", "postMessage", "print",
"prompt", "releaseEvents", "requestAnimationFrame", "resizeBy", "resizeTo",
"screen", "screenX", "screenY", "scroll", "scrollBy", "scrollByLines",
"scrollByPages", "scrollMaxX", "scrollMaxY", "scrollTo", "scrollX", "scrollY",
"scrollbars", "self", "sessionStorage", "setInterval", "setResizable",
"setTimeout", "showModalDialog", "sidebar", "sizeToContent", "speechSynthesis",
"status", "statusbar", "stop", "toolbar", "top", "updateCommands", "window"]
```

`window` is the top-level `this`.

`document === window.document`

Das **navigator**-Objekt vs. Das Navigator-Interface



https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Manipulating_documents

Vorsicht: automatische Globalisierung !

Alles was nicht deklariert wird, wird automatisch zur Property von **window**.

var, let, const vergessen

```
<script>
  function sum(max){
    for (i = 0; i<max; i++){
      x += i;
    }
    return x;
  }
  x = 1;
  sum(10);
  console.log( x ); // 46
</script>
```



Generelle Schnittstelle für Werte in Formularen: Form values ⇒ **value**-Property in HTMLElement

HTML:

```
<input type="text" name="name" id="uniqueID" value="value" />
```

JS:

```
var nameValue = document.getElementById("uniqueID").value;
```



https://stackoverflow.com/questions/3547035/javascript-getting-html-form-values

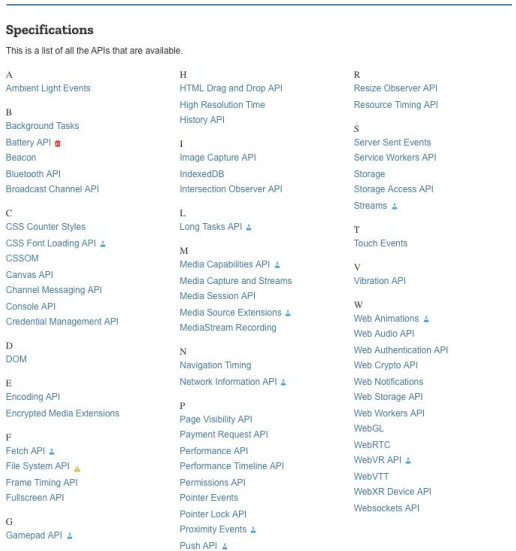
Ausblick: HTML DOM API interfaces ⊂ Web APIs

1. HTML Element interfaces
2. Web app and browser integration interfaces
3. Form support interfaces
4. Canvas and image interfaces
5. Media interfaces
6. Drag and drop interfaces
7. Page history interfaces
8. Web Components interfaces
9. Miscellaneous and supporting interfaces
10. Interfaces belonging to other APIs
    a. Web storage interfaces
    b. Web Workers interfaces
    c. WebSocket interfaces
    d. Server-sent events interfaces

https://developer.mozilla.org/en-US/docs/Web/API/HTML_DOM_API

Ausblick: Web APIs auf dem Stand von 2019

- **API (HTML or XML page) = DOM + JS (scripting language)**
- **66 Specifications**
- **922 Interfaces**



Specifications

This is a list of all the APIs that are available.

A
Ambient Light Events

B
Background Tasks
Battery API
Beacon
Bluetooth API
Broadcast Channel API

C
CSS Counter Styles
CSS Font Loading API
CSSOM
Canvas API
Channel Messaging API
Console API
Credential Management API

D
DOM

E
Encoding API
Encrypted Media Extensions

F
Fetch API
File System API
Frame Timing API
Fullscreen API

G
Gamepad API

H
HTML Drag and Drop API
High Resolution Time
History API

I
Image Capture API
IndexedDB
Intersection Observer API

L
Long Tasks API

M
Media Capabilities API
Media Capture and Streams
Media Session API
Media Source Extensions
MediaStream Recording

N
Navigation Timing
Network Information API

P
Page Visibility API
Payment Request API
Performance API
Performance Timeline API
Permissions API
Pointer Events
Pointer Lock API
Proximity Events
Push API

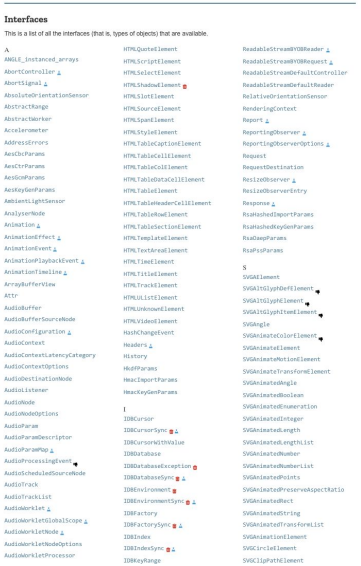
R
Resize Observer API
Resource Timing API

S
Server Sent Events
Service Workers API
Storage
Storage Access API
Streams

T
Touch Events

V
Vibration API

W
Web Animations
Web Audio API
Web Authentication API
Web Crypto API
Web Notifications
Web Storage API
Web Workers API
WebGL
WebRTC
WebVR API
WebVTT
WebXR Device API
Websockets API



Interfaces

This is a list of all the interfaces (that is, types of objects) that are available.

A
ANGLE_instanced_arrays
AbortController
AbortSignal
AbsoluteOrientationSensor
AbstractRange
AbstractWorker
Accelerometer
AddressErrors
AesCbcParams
AesCtrParams
AesGcmParams
AesKeyGenParams
AmbientLightSensor
AnalyserNode
Animation
AnimationEffect
AnimationEvent
AnimationPlaybackEvent
AnimationTimeline
ArrayBufferView
Attr
AudioBuffer
AudioBufferSourceNode
AudioConfiguration
AudioContext
AudioContextLatencyCategory
AudioContextOptions
AudioDestinationNode
AudioListener
AudioNode
AudioNodeOptions
AudioParam
AudioParamDescriptor
AudioParamMap
AudioProcessingEvent
AudioScheduledSourceNode
AudioTrack
AudioTrackList
AudioWorklet
AudioWorkletGlobalScope
AudioWorkletNode
AudioWorkletNodeOptions
AudioWorkletProcessor

HTMLQuoteElement
HTMLScriptElement
HTMLSelectElement
HTMLShadowElement
HTMLSlotElement
HTMLSourceElement
HTMLSpanElement
HTMLStyleElement
HTMLTableCaptionElement
HTMLTableCellElement
HTMLTableColElement
HTMLTableDataCellElement
HTMLTableElement
HTMLTableHeaderCellElement
HTMLTableRowElement
HTMLTableSectionElement
HTMLTemplateElement
HTMLTextAreaElement
HTMLTimeElement
HTMLTitleElement
HTMLTrackElement
HTMLUListElement
HTMLUnknownElement
HTMLVideoElement
HashChangeEvent
Headers
History
HkdfParams
HmacImportParams
HmacKeyGenParams

I
IDBCursor
IDBCursorSync
IDBCursorWithValue
IDBDatabase
IDBDatabaseException
IDBDatabaseSync
IDBEnvironment
IDBEnvironmentSync
IDBFactory
IDBFactorySync
IDBIndex
IDBIndexSync
IDBKeyRange

ReadableStreamBYOBReader
ReadableStreamBYOBRequest
ReadableStreamDefaultController
ReadableStreamDefaultReader
RelativeOrientationSensor
RenderingContext
Report
ReportingObserver
ReportingObserverOptions
Request
RequestDestination
ResizeObserver
ResizeObserverEntry
Response
RsaHashedImportParams
RsaHashedKeyGenParams
RsaOaepParams
RsaPssParams

S
SVGAElement
SVGAltGlyphDefElement
SVGAltGlyphElement
SVGAltGlyphItemElement
SVGAngle
SVGAnimateColorElement
SVGAnimateElement
SVGAnimateMotionElement
SVGAnimateTransformElement
SVGAnimatedAngle
SVGAnimatedBoolean
SVGAnimatedEnumeration
SVGAnimatedInteger
SVGAnimatedLength
SVGAnimatedLengthList
SVGAnimatedNumber
SVGAnimatedNumberList
SVGAnimatedPoints
SVGAnimatedPreserveAspectRatio
SVGAnimatedRect
SVGAnimatedString
SVGAnimatedTransformList
SVGAnimationElement
SVGCircleElement
SVGClipPathElement



ChildNode
Client
Clients
Clipboard
ClipboardEvent
CloseEvent
Comment
CompositionEvent
ConstantSourceNode
ConstrainBoolean
ConstrainDOMString
ConstrainDouble
ConstrainLong
ConvolverNode
Coordinates
CountQueuingStrategy
CrashReportBody
Credential
CredentialsContainer
Crypto
CryptoKey
CryptoKeyPair
CustomElementRegistry
CustomEvent

D
DOMConfiguration
DOMError
DOMException
DOMHighResTimeStamp
DOMImplementation
DOMImplementationList
DOMLocator
DOMMatrix
DOMMatrixReadOnly
DOMObject
DOMParser
DOMPoint
DOMPointInit
DOMPointReadOnly
DOMQuad
DOMRect
DOMRectReadOnly
DOMString
DOMStringList

MediaQueryListListener
MediaRecorder
MediaRecorderErrorEvent
MediaSession
MediaSettingsRange
MediaSource
MediaStream
MediaStreamAudioDestinationNode
MediaStreamAudioSourceNode
MediaStreamAudioSourceOptions
MediaStreamConstraints
MediaStreamEvent
MediaStreamTrack
MediaStreamTrackAudioSourceNode
MediaStreamTrackAudioSourceOptions
MediaStreamTrackEvent
MediaTrackConstraints
MediaTrackSettings
MediaTrackSupportedConstraints
MerchantValidationEvent
MessageChannel
MessageEvent
MessagePort
Metadata
MimeType
MimeTypeArray
MouseEvent
MouseScrollEvent
MouseWheelEvent
MutationEvent
MutationObserver
MutationObserverInit
MutationRecord

N
NameList
NamedNodeMap
NavigationPreloadManager
Navigator
NavigatorConcurrentHardware
NavigatorGeolocation
NavigatorID
NavigatorLanguage
NavigatorOnLine
NavigatorPlugins

SVGSetElement
SVGSolidcolorElement
SVGStopElement
SVGStringList
SVGStylable
SVGStyleElement
SVGSwitchElement
SVGSymbolElement
SVGTRefElement
SVGTSpanElement
SVGTests
SVGTextContentElement
SVGTextElement
SVGTextPathElement
SVGTextPositioningElement
SVGTitleElement
SVGTransform
SVGTransformList
SVGTransformable
SVGURIReference
SVGUnitTypes
SVGUseElement
SVGVKernElement
SVGViewElement
SVGZoomAndPan
Screen
ScreenOrientation
ScriptProcessorNode
ScrollToOptions
SecurityPolicyViolationEvent
Selection
Sensor
SensorErrorEvent
ServiceWorker
ServiceWorkerContainer
ServiceWorkerGlobalScope
ServiceWorkerMessageEvent
ServiceWorkerRegistration
ServiceWorkerState
ShadowRoot
SharedWorker
SharedWorkerGlobalScope
Slotable
SourceBuffer

https://developer.mozilla.org/en-US/docs/Web/API

Ausblick: Das JavaScript-Zeitalter

Atwood’s Law: any application that can be written in JavaScript, will eventually be written in JavaScript.

https://blog.codinghorror.com/the-principle-of-least-power/

Ausblick: Das JavaScript-Zeitalter

[We need to shift] our view of the server from a document courier (HTML Age), or a template renderer (LAMP Age), to a function and data shipper. The principal role of the server is to ship an **application** to the client (**JavaScript**), along with **data** (JSON), and let the client weave those into a **DOM**. [...]

The JavaScript age brings us closer to a web that is not a global digital library, but a **global digital nervous system**, whose implications we are only beginning to apprehend.

https://metamarkets.com/2011/node-js-and-the-javascript-age/

Ausblick: Das JavaScript-Zeitalter

